

# Sigma Protocols & Fiat Shamir Transform & Fischlin Transform

Tejas Pujari

December 2025

## 1 Identification Protocols

In this section we will introduce the foundations of interactive protocol and possible attack scenarios and thereafter move towards the constructions.

**Identification protocol:** An identification protocol is a triple  $I = (G, P, V)$

- $G$  : is a probabilistic key generation algorithm that takes no input but outputs the pair  $(vk, sk)$  namely verification key and secret key.
- $P$  : is an interactive algorithm called prover who takes input  $sk$
- $V$  : the verifier interactive algorithm who takes input the  $vk$

We require that when  $P(sk)$  interacts with  $V(vk)$  the output is always accept with probability 1.

Note that in this type of protocols the major problem that motivates above construction is impersonation attacks. That is someone possibly cheater wants to impersonate on behalf of honest person without valid secret. Majorly the threat scenario is classified in three attacks which we will be define shortly.

### 1.1 Direct Attacks

In short this type of impersonation is clean and straight that can a malicious party directly impersonates a honest party without any extra information.

**Attack Game: Secure identification against direct attacks:** For a given identification protocol  $I = (G, P, V)$  and given adversary  $A$

- Key generation phase: The challenger runs  $((vk, sk) \leftarrow G())$  and sends  $vk$  to  $A$ .
- Impersonation attempt: The challenger and  $A$  now interact with the challenger following the verifier algorithm and  $A$  playing the role of prover without  $sk$  but not necessarily follow the provers algorithm i.e try to do strategies to convince the verifier to accept.

We say that adversary wins the game if  $V$  outputs accept at the end of the interaction. We define  $A$  advantage against the protocol  $I$  as  $IDadv[A, I]$  is the probability that  $A$  wins the game.

**The protocol  $I$  is secure against the direct attack if  $IDadv[A, I]$  is negligible.**

## 1.2 Evaesdropping attack

In short this type of impersonation is same as direct attack but with a extra leverage to the adversary that he can eavesdrop the communication of the protocol and once he is done with that he can start attacking using the information he got during eavesdrop phase.

**Attack Game: Secure identification against eavesdrop attacks:** For a given identification protocol  $I = (G, P, V)$  and given adversary  $A$

- Key generation phase: The challenger runs  $((vk, sk) \leftarrow G())$  and sends  $vk$  to  $A$ .
- Eavesdrop Phase: The adversary request some number  $Q$ , of transcripts for the protocol to the challenger. Challenger then compiles by running the interaction between Prover and verifier a total of  $Q$  times with each time  $P$  is initialized with  $sk$  and  $V$  with  $vk$ . Challenger send these transcripts  $T_1, T_2, \dots, T_Q$  to the adversary.
- Impersonation attempt: The challenger and  $A$  now interact with the challenger following the verifier algorithm and  $A$  playing the role of prover without  $sk$  but not necessarily follow the provers algorithm i.e try to do strategies to convince the verifier to accept.

We say that adversary wins the game if  $V$  outputs accept at the end of the interaction. We define  $A$  advantage against the protocol  $I$  as  $ID2adv[A, I]$  is the probability that  $A$  wins the game.

**The protocol  $I$  is secure against the direct attack if  $ID2adv[A, I]$  is negligible.**

### 1.3 Active attacks

This is the last and most crucial attack in the regime of identification protocol where adversary have the maximum power.

**Attack Game: Secure identification against active attacks:** For a given identification protocol  $I = (G, P, V)$  and given adversary  $A$

- Key generation phase: The challenger runs  $((vk, sk) \leftarrow G())$  and sends  $vk$  to  $A$ .
- Active Probing Phase: The adversary request to interact with the prover. The challenger compiles by interacting with the adversary in an ID protocol with the challenger playing role of the prover by running algorithm  $P$  initialized with  $sk$ . The adversary plays the role of verifier the adversary may interact concurrently with many instances of prover these interactions maybe interleaved with each other.
- Impersonation attempt: The challenger and  $A$  now interact with the challenger following the verifier algorithm and  $A$  playing the role of prover without  $sk$  but not necessarily follow the provers algorithm i.e try to do strategies to convince the verifier to accept.

We say that adversary wins the game if  $V$  outputs accept at the end of the interaction. We define  $A$  advantage against the protocol  $I$  as  $ID3adv[A, I]$  is the probability that  $A$  wins the game.

**The protocol  $I$  is secure against the direct attack if  $ID3adv[A, I]$  is negligible.**

## 2 Schnorr's Identification Protocol

Now after describing the attack scenarios lets start to build some protocols that will give security against above attacks. In this text we will mostly talk about direct and eavesdrop attacks and not about active attacks.

### Schnorr's Protocol

- Keygen():  $G$  is a cyclic group of prime order.  $\alpha \in Z_q, g^\alpha \in G$   
 $(sk, vk) \leftarrow (\alpha, g^\alpha)$
- The protocol between  $P(\alpha)$  and  $V(g^\alpha)$ 
  1.  $P$  computes  $\alpha_t \leftarrow Z_q, u_t \leftarrow g^{\alpha_t}$ , and send  $u_t$  to  $V$ .

2. V computes  $c \leftarrow C$  and send to P.  $C \subset \mathbb{Z}_q$  called as challenge space.
3. P computes  $\alpha_z = \alpha_t + \alpha \cdot c$  and send to V.
4. V checks if  $g^{\alpha_z} = u_t \cdot (vk)^c$

**Theorem:** Under the Discrete Log assumption for group  $G$  and assuming  $N = |C|$  is super polynomial. Schnorr identification protocol is secure against direct attacks.

In particular suppose there is an efficient adversary  $A$  attacking  $I_{sch}$  via a direct attack defined above with advantage  $\epsilon = ID1adv[A, I_{sch}]$ . Then there exist an efficient DL adversary  $B$  who will break the discrete log with non negligible advantage.

**Proof idea** Suppose that  $\mathcal{A}$  has advantage  $\epsilon$  in attacking  $I_{sch}$  as in . In this game, the challenger generates the verification key  $u = g^\alpha$ . In his impersonation attempt, the adversary  $\mathcal{A}$  generates the first flow  $u_t$  of the protocol using some arbitrary adversarial strategy. Now, to succeed,  $\mathcal{A}$  must reply to a random challenge  $c \in \mathcal{C}$  with a valid response  $\alpha_z$  that satisfies  $g^{\alpha_z} = u_t \cdot u^c$ . Intuitively, if  $\mathcal{A}$  can generate a valid response to one such random challenge with probability  $\epsilon$ , it should be able to generate a valid response to two random challenges with probability  $\epsilon^2$ . Making this intuition rigorous requires a somewhat technical argument that will be presented in a lemma below.

So here is how we can use  $\mathcal{A}$  to compute the discrete logarithm of a random  $u \in G$ . We use  $u$  as the verification key in  $I_{sch}$ , and let  $\mathcal{A}$  generate the first flow  $u_t$  of the protocol. We then supply a random challenge  $c$  to  $\mathcal{A}$  and hope that  $\mathcal{A}$  generates a valid response  $\alpha_z$ . If this happens, we “rewind”  $\mathcal{A}$ ’s internal state back to the point just after which it generated  $u_t$ , and then supply  $\mathcal{A}$  with another random challenge  $c'$ , and hope that  $\mathcal{A}$  generates another valid response  $\alpha'_z$ .

If all of this happens, then we obtain two accepting conversations  $(u_t, c, \alpha_z)$  and  $(u_t, c', \alpha'_z)$  for a given verification key  $u$  and with matching first flows  $u_t$ . Moreover, with overwhelming probability, we have  $c' \neq c$ . Given this information, we can easily compute  $\text{Dlog}_g(u)$ . Indeed, since both conversations are accepting, we have the two equations

$$g^{\alpha_z} = u_t \cdot u^c \quad \text{and} \quad g^{\alpha'_z} = u_t \cdot u^{c'}.$$

Dividing the first equation by the second, the  $u_t$ ’s cancel, and we have

$$g^{\Delta\alpha_z} = u^{\Delta c}, \quad \text{where} \quad \Delta\alpha_z := \alpha_z - \alpha'_z, \quad \Delta c := c - c'.$$

Since  $\Delta c \neq 0$  and the group order  $q$  is prime, the inverse  $1/\Delta c$  exists in  $\mathbb{Z}_q$ . We can now raise both sides of the previous equation to the power  $1/\Delta c$ , obtaining

$$g^{\Delta\alpha_z/\Delta c} = u.$$

Therefore, we can efficiently compute  $\text{Dlog}_g(u)$  as  $\Delta\alpha_z/\Delta c$ .

**Note:** The technique used above is famously known as rewinding technique which is the crux of the extraction for the DL witness. We will see later that this notion is quite widely used for almost everywhere in the reduction proofs for protocols.

### 3 Honest Verifier Zero knowledge and security against eavesdropping attack

We have shown that Schnorr's identification protocol is secure against *direct attacks*, under the DL assumption. In fact, under the same assumption, we can show that Schnorr's identification protocol is secure against *eavesdropping attacks* as well. Now, in an eavesdropping attack, the adversary obtains  $vk$  and a list of transcripts—conversations between  $P$  (on input  $sk$ ) and  $V$  (on input  $vk$ ). The idea is to show that these conversations do not help the adversary, because the adversary could have efficiently generated these conversations by himself, given  $vk$  (but not  $sk$ ). If we can show this, then we are done. Indeed, suppose  $\mathcal{A}$  is an adversary whose advantage in carrying out a successful impersonation via an eavesdropping attack is non-negligible. Then we replace  $\mathcal{A}$  by another adversary  $\mathcal{B}$ , that works the same as  $\mathcal{A}$ , except that  $\mathcal{B}$  generates the transcripts by himself, instead of obtaining them from his challenger. Thus,  $\mathcal{B}$  carries out a direct attack, but has the same advantage as  $\mathcal{A}$  in carrying out a successful impersonation.

We shall develop this idea in a more general way, introducing the notion of *honest verifier zero knowledge*.

**Definition** Let  $\mathcal{I} = (G, P, V)$  be an identification protocol. We say that  $\mathcal{I}$  is *honest verifier zero knowledge*, or *HVZK* for short, if there exists an efficient probabilistic algorithm  $\text{Sim}$  (called a *simulator*) such that for all possible outputs  $(vk, sk)$  of  $G$ , the output distribution of  $\text{Sim}$  on input  $vk$  is *identical* to the distribution of a transcript of a conversation between  $P$  (on input  $sk$ ) and  $V$  (on input  $vk$ ).

**Theorem :** If an identification protocol  $\mathcal{I}$  is secure against direct attacks, and is HVZK, then it is secure against eavesdropping attacks.

In particular, if  $\mathcal{I}$  is HVZK with simulator  $\text{Sim}$ , then for every impersonation adversary  $\mathcal{A}$  that attacks  $\mathcal{I}$  via an eavesdropping attack, obtaining up to  $Q$  transcripts, there is an adversary  $\mathcal{B}$  that attacks  $\mathcal{I}$  via a direct attack, as in where  $\mathcal{B}$  is an elementary wrapper around  $\mathcal{A}$  (and where  $\mathcal{B}$  runs  $\text{Sim}$  at most  $Q$  times), such that

$$\text{ID2adv}[\mathcal{A}, \mathcal{I}] = \text{ID1adv}[\mathcal{B}, \mathcal{I}].$$

*Proof.*  $\mathcal{B}$  works the same as  $\mathcal{A}$ , except that instead of obtaining transcripts from its challenger, it generates the transcripts itself using  $\text{Sim}$ .

Let us now return to Schnorr's identification protocol.

**Theorem 19.4.** Schnorr's identification protocol is HVZK.

*Proof.* The idea is that in generating a simulated conversation  $(u_t, c, \alpha_z)$ , we do not need to generate the messages of the conversation in the given order, as in a real conversation between  $P$  and  $V$ . Indeed, our simulator  $\text{Sim}$  generates the messages in reverse order. On input  $vk = u$ , the simulator  $\text{Sim}$  computes

$$\alpha_z \xleftarrow{\$} \mathbb{Z}_q, \quad c \xleftarrow{\$} \mathcal{C}, \quad u_t \leftarrow g^{\alpha_z} / u^c,$$

and outputs the conversation  $(u_t, c, \alpha_z)$ .

Now we argue that the output of  $\text{Sim}$  on input  $vk = u$  has the right distribution.

determined by the equation  $g^{\alpha_z} = u_t \cdot u^c$ . It should be clear that this is the same as the output distribution of the simulator.  $\square$

As a corollary, we immediately obtain:

**Theorem 19.5.** *If Schnorr's identification protocol is secure against direct attacks, then it is also secure against eavesdropping attacks.*

In particular, for every impersonation adversary  $\mathcal{A}$  that attacks  $\mathcal{I}_{\text{Sch}}$  via an eavesdropping attack, as in , there is an adversary  $\mathcal{B}$  that attacks  $\mathcal{I}_{\text{Sch}}$  via a direct attack, as in , where  $\mathcal{B}$  is an elementary wrapper around  $\mathcal{A}$ , such that

$$\text{ID2adv}[\mathcal{A}, \mathcal{I}_{\text{Sch}}] = \text{ID1adv}[\mathcal{B}, \mathcal{I}_{\text{Sch}}].$$

**Why this is not contradictory.**

- At first sight, the results about Schnorr's protocol may seem **counter-intuitive**: it is *hard* to carry out an impersonation attack given only  $vk$ , yet *easy* to generate transcripts knowing only  $vk$ .
- The key difference is **interaction and timing**. In an impersonation attack:
  - the verifier  $V$  is **actively involved**;
  - the adversary (as prover) must send the first message  $u_t$  **before** seeing the challenge  $c$  sampled by  $V$ .
- A simulator, by contrast, can **choose the message order**. In the HVZK simulator:
  - it first samples  $c$  and  $\alpha_z$ ;
  - then it computes  $u_t$  so that  $(u_t, c, \alpha_z)$  is an accepting conversation.
- These results also highlight a potential **insecurity if the challenge space is too small**. If  $|\mathcal{C}|$  were small, an adversary could:
  - use the simulator to prepare an accepting transcript  $(u_t, c, \alpha_z)$ ;

- send  $u_t$  to  $V$  and hope that the verifier’s challenge equals the precomputed  $c$ ;
  - if this happens, respond with  $\alpha_z$  and make  $V$  accept.
- In that case, Schnorr’s identification protocol could be broken with **advantage about**  $1/|\mathcal{C}|$ . Therefore, the challenge space size  $|\mathcal{C}|$  must be taken **super-polynomially large** (in the security parameter) to ensure security.

## 4 Sigma Protocols

Schnorr’s identification protocol is a special case of an incredibly useful class of protocol called as sigma protocols. In this section we will see how sigma protocols are used to build new identification and signature schemes.

**Effective Relation:** An effective relation is a binary relation  $R \subset X \times Y$  where  $X, Y, R$  are efficiently recognizable finite sets. Elements of  $Y$  are called as statements and if  $(x, y) \in R$  then  $x$  is called as the witness for  $y$ .

**Sigma Protocol** Definition (Sigma protocol). Let  $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$  be an effective relation. A *Sigma protocol* for  $\mathcal{R}$  is a pair  $(P, V)$ .

- $P$  is an interactive protocol algorithm called the *prover*, which takes as input a witness-statement pair  $(x, y) \in \mathcal{R}$ .
- $V$  is an interactive protocol algorithm called the *verifier*, which takes as input a statement  $y \in \mathcal{Y}$ , and which outputs *accept* or *reject*.
- $P$  and  $V$  are structured so that an interaction between them always works as follows:
  - To start the protocol,  $P$  computes a message  $t$ , called the *commitment*, and sends  $t$  to  $V$ ;
  - Upon receiving  $P$ ’s commitment  $t$ ,  $V$  chooses a *challenge*  $c$  at random from a finite *challenge space*  $\mathcal{C}$ , and sends  $c$  to  $P$ ;
  - Upon receiving  $V$ ’s challenge  $c$ ,  $P$  computes a *response*  $z$ , and sends  $z$  to  $V$ ;
  - Upon receiving  $P$ ’s response  $z$ ,  $V$  outputs either *accept* or *reject*, which must be computed strictly as a function of the statement  $y$  and the conversation  $(t, c, z)$ . In particular,  $V$  does not make any random choices other than the selection of the challenge— all other computations are completely deterministic.

We require that for all  $(x, y) \in \mathcal{R}$ , when  $P(x, y)$  and  $V(y)$  interact with each other,  $V(y)$  always outputs *accept*.

**Special soundness** We next define a critical security property for Sigma protocols, which is called *special soundness*.

**Definition (Special soundness).** Let  $(P, V)$  be a Sigma protocol for  $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ . We say that  $(P, V)$  provides *special soundness* if there is an efficient deterministic algorithm **Ext**, called a *witness extractor*, with the following property: whenever **Ext** is given as input a statement  $y \in \mathcal{Y}$ , and two accepting conversations  $(t, c, z)$  and  $(t, c', z')$ , with  $c \neq c'$ , algorithm **Ext** always outputs  $x \in \mathcal{X}$  such that  $(x, y) \in \mathcal{R}$  (i.e.,  $x$  is a witness for  $y$ ).

**Special honest verifier zero knowledge** We introduced the notion of honest verifier zero knowledge (HVZK) in Section 19.1.1 for identification protocols. We can easily adapt this notion to the context of Sigma protocols.

Let  $(P, V)$  be a Sigma protocol for  $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ . Intuitively, what we want to say is that for  $(x, y) \in \mathcal{R}$ , a conversation between  $P(x, y)$  and  $V(y)$  should not reveal anything about the witness  $x$ . Just as in Section 19.1.1, we will formalize this intuition by saying that we can efficiently simulate conversations between  $P(x, y)$  and  $V(y)$  without knowing the witness  $x$ . However, we will add a few extra requirements, which will streamline some constructions and applications.

**Definition (Special HVZK).** Let  $(P, V)$  be a Sigma protocol for  $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$  with challenge space  $\mathcal{C}$ . We say that  $(P, V)$  is *special honest verifier zero knowledge*, or *special HVZK*, if there exists an efficient probabilistic algorithm **Sim** (called a *simulator*) that takes as input  $(y, c) \in \mathcal{Y} \times \mathcal{C}$ , and satisfies the following properties:

- (i) for all inputs  $(y, c) \in \mathcal{Y} \times \mathcal{C}$ , algorithm **Sim** always outputs a pair  $(t, z)$  such that  $(t, c, z)$  is an accepting conversation for  $y$ ;
- (ii) for all  $(x, y) \in \mathcal{R}$ , if we compute

$$c \xleftarrow{\$} \mathcal{C}, \quad (t, z) \xleftarrow{\$} \mathbf{Sim}(y, c),$$

then  $(t, c, z)$  has the same distribution as that of a transcript of a conversation between  $P(x, y)$  and  $V(y)$ .

The reader should take note of a couple of features of this definition. First, the simulator takes the challenge  $c$  as an additional input. Second, it is required that the simulator produce an accepting conversation even when the statement  $y$  does not have a witness. These two properties are the reason for the word “special” in “special HVZK.”

**Example** Continuing with Example, we can easily verify that Schnorr’s Sigma protocol is special HVZK. Indeed, the simulator in the proof of Theorem 19.4 is easily adapted to the present setting. On input  $u \in \mathbb{G}$  and  $c \in \mathcal{C}$ , the simulator computes

$$\alpha_z \xleftarrow{\$} \mathbb{Z}_q, \quad u_t \leftarrow g^{\alpha_z} / u^c,$$

and outputs the pair  $(u_t, \alpha_z)$ .

## 5 Identification and signatures from sigma protocols

By mimicking the schnorr construction we can easily convert any sigma protocol into a corresponding identification scheme and signature scheme.

Suppose we have a Sigma protocol  $(P, V)$  for a relation  $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ . In addition to  $P$  and  $V$ , we need a *key generation algorithm* for  $\mathcal{R}$ . This is a probabilistic algorithm  $G$  that generates a public-key/secret-key pair  $(pk, sk)$ , where  $pk = y$  and  $sk = (x, y)$  for some  $(x, y) \in \mathcal{R}$ .

To get secure identification and signature schemes we need the following “one-wayness” property: given a public key  $pk = y \in \mathcal{Y}$  output by  $G$ , it should be hard to compute  $\hat{x} \in \mathcal{X}$  such that  $(\hat{x}, y) \in \mathcal{R}$ . This notion is made precise by the following attack game.

**Attack Game (One-way key generation).** Let  $G$  be a key generation algorithm for  $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ . For a given adversary  $\mathcal{A}$ , the attack game runs as follows:

- The challenger runs  $(pk, sk) \xleftarrow{\$} G()$ , and sends  $pk = y$  to  $\mathcal{A}$ ;
- $\mathcal{A}$  outputs  $\hat{x} \in \mathcal{X}$ .

We say that the adversary wins the game if  $(\hat{x}, y) \in \mathcal{R}$ . We define  $\mathcal{A}$ ’s advantage with respect to  $G$ , denoted  $\text{OWadv}[\mathcal{A}, G]$ , as the probability that  $\mathcal{A}$  wins the game.  $\square$

**Definition** We say that a key generation algorithm  $G$  is *one way* if for all efficient adversaries  $\mathcal{A}$ , the quantity  $\text{OWadv}[\mathcal{A}, G]$  is negligible.

**Example** For the Schnorr Sigma protocol (Example 19.1), the most natural key generation algorithm computes  $\alpha \xleftarrow{\$} \mathbb{Z}_q$  and  $u \leftarrow g^\alpha \in \mathbb{G}$ , and outputs  $pk := u$  and  $sk := (\alpha, u)$ . It is clear that this key generation algorithm is one-way under the DL assumption.  $\square$

**Below theorems are easy to prove following the schnorr proofs we have seen above.** Theorem 19.14. Let  $(P, V)$  be a Sigma protocol for an effective relation  $\mathcal{R}$  with a large challenge space. Let  $G$  be a key generation algorithm for  $\mathcal{R}$ . If  $(P, V)$  provides special soundness and  $G$  is one-way, then the identification scheme  $\mathcal{I} := (G, P, V)$  is secure against direct attacks.

Theorem 19.15. Let  $(P, V)$  be a Sigma protocol for an effective relation  $\mathcal{R}$ . Let  $G$  be a key generation algorithm for  $\mathcal{R}$ . If the identification protocol  $\mathcal{I} = (G, P, V)$  is secure against direct attacks, and  $(P, V)$  is special HVZK, then  $\mathcal{I}$  is also secure against eavesdropping attacks.

In particular, for every impersonation adversary  $\mathcal{A}$  that attacks  $\mathcal{I}$  via an eavesdropping attack, as in , there is an adversary  $\mathcal{B}$  that attacks  $\mathcal{I}$  via a direct attack, as in , where  $\mathcal{B}$  is an elementary wrapper around  $\mathcal{A}$ , such that

$$\text{ID2adv}[\mathcal{A}, \mathcal{I}] = \text{ID1adv}[\mathcal{B}, \mathcal{I}].$$

## 6 Fiat Shamir Heuristics for signatures

We can create signatures using sigma protocols defined above if we recall signatures are a type of authentication schemes where the signer uses his own secret key to sign the message and then anyone in the world can verify the signature using signer's public key. Sigma protocols above are interactive by design atleast for the challenge phase from the verifier we need a fresh challenge which deduce the interaction. If we make this non-interactive that means if prover can do this on behalf of the verifier without cheating then we are done. This is exactly the idea of Fiat-shamir transform.

- a sigma protocol  $(P,V)$  for a relation  $R$  we assume that the conversations are of the form  $(t, c, z)$  where  $t \in T$   $c \in C$   $z \in Z$
- a key generation algorithm  $G$  for  $R$ ;
- a hash function  $H : M \times T \rightarrow C$  which will be modeled as a random oracle; the set  $M$  will be the message space of the signature scheme.

The Fiat shamir signature scheme derived from  $G$  and  $(P,V)$  works as follows:

- The key generation algorithm is  $G$ , so public key  $Pk = y$ , where  $y \in Y$  and a secret key is of the form  $sk = (x, y) \in R$
- To sign a message  $m \in M$  using a secret key  $sk = (x, y)$ , the signing algorithm runs as follows:
  - it starts the prover  $P(x, y)$ , obtaining the commitment  $t \in T$
  - It computes a challenge  $c \leftarrow H(m, t)$
  - finally it feeds  $c$  to the prover to obtain the response  $z$  and outputs the signature  $\sigma = (t, z) \in T \times Z$
  - Verification of signature: On input  $(m, t, z)$  verifier do  $H(m, t) \rightarrow c$  check if whether  $(t, c, z)$  is an accepting conversation for  $y$ .

**definition** [Unpredictable commitments] Let  $(P, V)$  be a  $\Sigma$ -protocol for  $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ , and suppose that all conversations  $(t, c, z)$  lie in  $\mathcal{T} \times \mathcal{C} \times \mathcal{Z}$ . We say that  $(P, V)$  has  $\delta$ -unpredictable commitments if for every  $(x, y) \in \mathcal{R}$  and  $\hat{t} \in \mathcal{T}$ , with probability at most  $\delta$ , an interaction between  $P(x, y)$  and  $V(y)$  produces a conversation  $(t, c, z)$  with  $t = \hat{t}$ . We say that  $(P, V)$  has *unpredictable commitments* if it has  $\delta$ -unpredictable commitments for negligible  $\delta$ .

**Theorem 1.** *If  $H$  is modeled as a random oracle, the identification scheme  $\mathcal{I} = (G, P, V)$  is secure against eavesdropping attacks, and  $(P, V)$  has unpredictable commitments, then the Fiat-Shamir signature scheme  $\mathcal{S}$  derived from  $G$  and  $(P, V)$  is secure.*

*In particular, let  $\mathcal{A}$  be an adversary attacking  $\mathcal{S}$  as in the random oracle version of above attack game. Moreover, assume that  $\mathcal{A}$  issues at most  $Q_s$  signing queries and  $Q_{ro}$  random oracle queries, and that  $(P, V)$  has  $\delta$ -unpredictable commitments. Then there exists a  $(Q_{ro} +$*

1)-impersonation adversary  $\mathcal{B}$  that attacks  $\mathcal{I}$  via an eavesdropping attack as in 19.1, where  $\mathcal{B}$  is an elementary wrapper around  $\mathcal{A}$ , such that

$$\text{SIG}^{roadv}[\mathcal{A}, \mathcal{S}] \leq Q_s (Q_s + Q_{ro} + 1) \delta + r\text{ID2adv}[\mathcal{B}, \mathcal{I}, Q_{ro} + 1].$$

We will prove the above theorem for the schnorr signature which is the secure instantiation of FS signature scheme designed above for instance:

- **Key Generation:**

- Let  $G$  be a group generation algorithm for a cyclic group of order  $q$  with generator  $g$ .
- The secret key is  $\alpha \in \mathbb{Z}_q$ ; the public key is  $u = g^\alpha$ .

- **Sigma Protocol  $(P, V)$ :**

- $(P, V)$  is a canonical  $\Sigma$ -protocol for the discrete logarithm relation with  $(u, \alpha)$  as key pair.

- **Schnorr Identification:** The identification scheme is  $\mathcal{I}_{\text{Sch}} = (G, P, V)$ .

- **Fiat–Shamir Signature Construction:**

- **KeyGen:** Output  $(\alpha, u)$  as above.
- **Signing:** To sign  $m \in \mathcal{M}$ :
  - \* Sample  $\alpha_t \in \mathbb{Z}_q$  and set  $t = g^{\alpha_t}$ .
  - \* Compute  $c = H(m, t)$  where  $H$  is a hash function modeled as a random oracle.
  - \* Compute  $\alpha_z = \alpha_t + \alpha \cdot c \pmod{q}$ .
  - \* Output signature  $\sigma = (t, \alpha_z)$ .
- **Verification:** On input  $(m, \sigma = (t, \alpha_z))$  and public key  $u$ :
  - \* Compute  $c = H(m, t)$ .
  - \* Accept if  $g^{\alpha_z} = t \cdot u^c$ ; otherwise, reject.

Now we will see the proof of above security of FS signature in random oracle model with schnorr setup. We do a slightly different reduction as a first step which is r-impersonation attack game which is defined as below

**r-impersonation eavesdropping attack game** . For a given identification protocol  $I = (G, P, V)$ , positive integer  $r$  and adversary  $A$  the attack game runs as follows. The key generation and eavesdropping phase is exactly same the only difference is that instead of 1 adversary is allowed to interact with  $r$  concurrent verifiers. The challenger itself plays the role of verifiers all of which uses same verification key as generated during the key generation phase. The adversary wins the game if it makes any of these verifiers output accept.

We define the advantage of adversary in the r-impersonation game denoted  $r\text{ID2adv}[A, I, r]$  The probability that  $A$  wins the game.

**Lemma:** Let  $I$  be an identification protocol. For every  $r$ -impersonation eavesdropping adversary  $A$  there exist a standard eavesdropping adversary  $B$  which is elementary wrapper around  $A$  such that,

$$rID2adv[A, I, r] \leq r \cdot ID2adv[B, I]$$

Now we are ready to prove the theorem in slightly different way.

**Theorem 2.** *If  $H$  is modeled as a random oracle, the identification scheme  $\mathcal{I}_{S\|\zeta} = (G, P, V)$  is secure against  $r$ -impersonation eavesdropping attacks, and  $(P, V)$  has unpredictable commitments, then the Fiat-Shamir Schnorr signature scheme  $\mathcal{S}$  derived from  $G$  and  $(P, V)$  is secure.*

*In particular, let  $\mathcal{A}$  be an adversary attacking  $\mathcal{S}$  as in the random oracle version of  $\text{Attack}_{FSig_{sch}}$ . Moreover, assume that  $\mathcal{A}$  issues at most  $Q_s$  signing queries and  $Q_{ro}$  random oracle queries, and that  $(P, V)$  has  $\delta$ -unpredictable commitments. Then there exists a  $(Q_{ro} + 1)$ -impersonation adversary  $\mathcal{B}$  that attacks  $\mathcal{I}$  via an eavesdropping attack as in  $\text{Attack}$  where  $\mathcal{B}$  is an elementary wrapper around  $\mathcal{A}$ , such that*

$$SIG^{ro}adv[\mathcal{A}, \mathcal{S}] \leq Q_s (Q_s + Q_{ro} + 1) \delta + rID2adv[\mathcal{B}, \mathcal{I}, Q_{ro} + 1].$$

**Proof:** We proceed with the game based security approach and design two games  $\text{Game0}$  is original signature game and  $\text{Game1}$  is slight modification of  $\text{Game0}$ .

### Game 0 Challenger

initialization

$$\alpha \leftarrow Z_q, \quad u \leftarrow g^\alpha$$

Initialize the empty random oracle array  $RO : M \times G \rightarrow C$  and  
 $Explicit : M \times G \rightarrow Z$

send public key to  $A$

**upon receiving the  $i$ -th signing query  $m_i \in M$**

$$\alpha_{ti} \leftarrow Z_q, \quad u_{ti} \leftarrow g^{\alpha_{ti}}, \quad c_i \leftarrow C$$

1 if  $(m_i, u_{ti}) \in \text{Domain}(RO)$  then  $c_i \leftarrow RO[m_i, u_{ti}]$

$$RO[m_i, u_{ti}] \leftarrow c_i$$

$$\alpha_{zi} \leftarrow \alpha_{ti} + \alpha \cdot c_i$$

send  $(u_{ti}, \alpha_{zi})$  to  $A$

**upon recieving j-th random oracle query**  $(\hat{m}_j, \hat{u}_j) \in M \times G$

if  $(\hat{m}_j, \hat{u}_j) \notin \text{Domain}(RO)$  then

$Ro[(\hat{m}_j, \hat{u}_j)] \leftarrow C, \text{ Explicit}[(\hat{m}_j, \hat{u}_j)] \leftarrow j$

send  $RO[(\hat{m}_j, \hat{u}_j)]$  to  $A$

upon recieving forgery attempt  $(m, y_t, \alpha_z)$  :

By assumption,  $(m, u_t) \in \text{Domain}(\text{Explicit})$

if  $g^{\alpha_z} = u_t \cdot u^c$  where  $c = RO[m, u_t]$

then output "win"

else output "lose"

**Proof sketch.** To simplify the analysis, assume that whenever  $\mathcal{A}$  outputs a forgery pair  $(m, \sigma)$ , where  $\sigma = (u_t, \alpha_z)$ , then  $\mathcal{A}$  has already explicitly queried the random oracle at the point  $(m, u_t)$ . If necessary, modify  $\mathcal{A}$  so this is the case, increasing its total number of random-oracle queries by at most 1, so the modified adversary makes at most  $Q_{ro} + 1$  queries in total.

We now define two attack games. Game 0 is the original signature-attack game, with  $H$  modeled as a random oracle. Game 1 is a slight modification. For  $j \in \{0, 1\}$ , let  $W_j$  denote the event that  $\mathcal{A}$  wins in Game  $j$ .

**Game 0.** As usual:

- The random oracle  $H$  is implemented using an associative array

$$RO : \mathcal{M} \times \mathcal{G} \rightarrow \mathcal{C}.$$

- An additional associative array

$$\text{Explicit} : \mathcal{M} \times \mathcal{G} \rightarrow \mathbb{Z}_q$$

keeps track of points  $(m, u)$  where the random oracle value was first chosen explicitly by the adversary, rather than implicitly by the signing algorithm.

The challenger responds to oracle queries as follows:

- **Signing query on  $m_i$ :**
  - Sample  $\alpha_{t_i} \leftarrow \mathbb{Z}_q$  and set  $u_{t_i} \leftarrow g^{\alpha_{t_i}}$ .
  - Generate a random value  $c_i \in \mathcal{C}$  for  $\text{Map}[m_i, u_{t_i}]$ .
  - If the test in line (marked (1)) detects that  $\text{Map}[m_i, u_{t_i}]$  was already defined, then the previously defined value is used instead of the random value.
- **Random-oracle query on  $(\hat{m}_j, \hat{u}_j)$ :**
  - If  $\text{Map}[\hat{m}_j, \hat{u}_j]$  is not yet defined by a previous signing or random-oracle query, then define it and additionally set  $\text{Explicit}[\hat{m}_j, \hat{u}_j] \leftarrow j$ .

Suppose now that  $\mathcal{A}$  outputs a forgery  $(m, u_t, \alpha_z)$ , and that  $m$  is different from all messages  $m_i$  used in signing queries. By the simplifying assumption,  $\mathcal{A}$  must have previously queried the random oracle at  $(m, u_t)$ . Hence  $(m, u_t)$  lies in the domain of **Explicit**, and if  $(u_t, \alpha_z)$  is a valid signature, the challenger outputs “win”. Therefore

$$\text{SIG}^{roadv}[\mathcal{A}, \mathcal{S}_{\text{Sch}}] \leq \Pr[W_0].$$

**Game 1.** Game 1 is identical to Game 0, except that the line marked (1) in Fig. 19.3 is deleted. By the Difference Lemma,

$$|\Pr[W_1] - \Pr[W_0]| \leq Q_s (Q_s + Q_{ro} + 1)/q.$$

Indeed, for the  $i$ -th signing query,  $u_{t_i}$  is uniformly distributed over  $\mathcal{G}$ . Using the union bound, the probability that the random oracle was previously queried at  $(m, u_{t_i})$  (directly by the adversary or indirectly via a previous signing query) is at most  $(Q_s + Q_{ro} + 1)/q$ . Applying the union bound over all signing queries yields the overall bound  $Q_s (Q_s + Q_{ro} + 1)/q$  for the probability that this event occurs for any signing query.

The point of this change is that in Game 1, a fresh random challenge is used for each signing query, exactly as an honest verifier would behave in Schnorr’s identification protocol. At this stage it is easy to construct an adversary  $\mathcal{B}$  that plays the  $r$ -impersonation eavesdropping attack game with  $r = Q_{ro} + 1$  against a challenger and internally simulates the challenger for  $\mathcal{A}$  in Game 1. Consequently,

$$\Pr[W_1] = r\text{ID2}[\mathcal{B}, \mathcal{I}_{\text{Sch}}, r].$$

Now we will describe the adversary  $\mathcal{B}$  who simulates the above game 0 for the adversary  $\mathcal{A}$ .

### Adversary $\mathcal{B}$ simulation

initialization

obtain the verification key  $u$  from the challenger.

Obtain eavesdropped conversations  $(u_{ti}, c_i, \alpha_{zi})$  for  $i = 1, 2, \dots, Q_s$  from challenger

initialize the empty associate array  $Ro : M \times G \rightarrow C$  and  
 $Explicit : M \times G \rightarrow Z$

send  $u$  to  $A$ .

**upon receiving the  $i$ th signing query  $m_i \in M$  from  $A$  :**

$Ro[m_i, u_{ti}] \leftarrow c_i$

send  $(u_{ti}, \alpha_{zi})$  to the adversary  $A$

upon receiving random oracle query  $(\hat{m}_j, \hat{u}_j)$ :

if  $(\hat{m}_j, \hat{u}_j) \notin \text{Domain}(Ro)$  then

initiate an impersonation attempt with the verifier  $V_j$ .

send  $\hat{u}_j$  to  $V_j$  who responds with a challenger  $\hat{c}_j$

$Ro[\hat{m}_j, \hat{u}_j] \leftarrow \hat{c}_j$ ,  $Explicit[(\hat{m}_j, \hat{u}_j)] \leftarrow j$

send  $Ro[\hat{m}_j, \hat{u}_j]$  to  $A$ .

upon receiving a forgery attempt  $(m, u_t, \alpha_z)$  :

- By assumption  $(m, u_t) \in \text{Domain}(Explicit)$

send the final message  $\alpha_z$  to  $V_j$  where  $j = Explicit[m, u_t]$

Putting it all together we get,

$$SIG^{roadv}[A, FSig_{sch}] \leq Q_s (Q_s + Q_{ro} + 1)/q + rID2adv[B, I_{sch}, r]$$

The above security bound is not tight recall that we can further reduce the r-impersonation to standard eavesdrop impersonation and standard eavesdrop impersonation to impersonation against direct attack and that to discrete log hardness using rewinding. So finally the total reduction of the above FS-signature scheme boils down to the witness extraction using rewinding. This implies that security reduction is strongly rely on random oracles and rewinding.

## 7 Multi-extractability: another notion of “proof of knowledge”

we noted that Sigma protocols such as Schnorr’s act as a “proof of knowledge”, meaning that a witness can be extracted from a prover using a particular rewinding technique. We used this rewinding technique to prove the security of Schnorr’s identification and signature schemes (and similar schemes based on more general Sigma protocols).

The notion of a proof of knowledge is often used as a step towards proving security of some candidate scheme (such as security of a signature scheme). We have not attempted to give a general definition for a “proof of knowledge” because the exact definition can depend to a certain degree on the application at hand, and on the exact statement one is trying to prove.

In this section we explore an interesting “proof of knowledge” property that Sigma protocols satisfy called *multi-extractability*. This property is also based on a rewinding technique, but the technique is different than the rewinding technique used to prove the security of Schnorr’s identification and signature schemes. We shall discuss some applications and limitations of multi-extractability.

Intuitively, multi-extractability means the following. Suppose that during an attack, an adversary is allowed to interact concurrently with many verifiers, supplying each verifier with a statement of the adversary’s choice, and at the end of the attack, the adversary makes some of these verifiers accept. Then we can “shake the adversary” (i.e., “rewind” or “re-run” the adversary) over and over again until it finally “spits out what it knows” (i.e., we extract witnesses for all statements supplied to the accepting verifiers).

### Multi-extractability for Sigma-protocols

**Setting.** Let  $\Pi$  be a  $\Sigma$ -protocol for an NP relation  $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$  over statements  $y \in \mathcal{X}$  with witnesses  $x \in \mathcal{W}$ . Conversations (transcripts) are triples  $(t, c, z)$ , where  $t \in \mathcal{T}$  is the commitment,  $c \in \mathcal{C}$  is the challenge, and  $z \in \mathcal{Z}$  is the response, and acceptance is publicly checkable from  $(y, t, c, z)$ .  $\Pi$  has special soundness: from two accepting transcripts sharing the same  $t$  but with  $c \neq c'$ , one can efficiently compute a witness for  $y$ . This standard property underlies extraction by rewinding.

**Verifier-oracle interface.** An adversary  $\mathcal{A}$  interacts with a verifier oracle that, on input a pair  $(y, t)$ , returns a uniformly random challenge  $c \leftarrow \mathcal{C}$ , after which  $\mathcal{A}$  can output a response  $z$ . Each such query is one “conversation attempt” on statement  $y$  with commitment  $t$ . Acceptance means  $(t, c, z)$  verifies for  $y$ . This oracle formalization is common in analyses of  $\Sigma$ -protocol extraction.

**B-bounded adversary.**  $\mathcal{A}$  is called  $B$ -bounded if in any execution it makes at most  $r \leq B$  verifier-oracle queries (i.e., at most  $B$  conversations across all statements). Bounding  $r$  lets the extractor’s runtime and number of rewinds scale polynomially in  $B$  and in  $1/\theta$  below. Intuitively,  $B$  caps concurrency/parallel sessions so that rewinding does not blow up.

**Prescribed failure probability.** Fix a target failure budget  $\theta \in (0, 1)$  chosen by the reduction. The multi-extractor is allowed to output **fail** with probability at most  $\theta$  (up to a negligible slack), while otherwise outputting witnesses for all accepting indices (defined next). The idea that extraction may deliberately allow a small failure probability is standard when controlling rewinding under many sessions.

**Initial execution and accepting indices.** Run  $\mathcal{A}$  once from start to finish with a verifier that answers with uniform challenges and provides fresh random bits wherever  $\mathcal{A}$  requests randomness. Record a transcript of all  $r \leq B$  verifier-oracle interactions:

$$((y_1, t_1), c_1), \dots, ((y_r, t_r), c_r)$$

together with  $\mathcal{A}$ ’s final output  $(J, \{z_j\}_{j \in J})$ , where  $J \subseteq \{1, \dots, r\}$  is the set of indices such that each conversation  $(t_j, c_j, z_j)$  is accepting for  $y_j$ . Only the oracle I/O and  $\mathcal{A}$ ’s final output are logged; the transcript omits the answers to  $\mathcal{A}$ ’s internal randomness requests so that they can be freshly resampled in re-runs.

**Multi-extraction experiment (game).** Given the bound  $B$  and the failure budget  $\theta$ , a multi-extractor **MEx** receives the initial transcript and may repeatedly re-run  $\mathcal{A}$  as follows. For each re-run, **MEx** chooses an index  $j \in J$  (the “rewinding target”) and restarts  $\mathcal{A}$  from the beginning, deterministically restoring  $\mathcal{A}$ ’s internal state up to the moment it issued the  $j$ -th verifier-oracle query in the initial run. From that point on, the re-run proceeds with: - A fresh random challenge to the  $j$ -th query, independent of the original  $c_j$ . - Fresh, independent random bits for any later randomness that  $\mathcal{A}$  requests. - Fresh random challenges for all verifier-oracle queries at indices  $> j$  that occur in the re-run. At the end of each re-run, **MEx** receives the new full transcript of oracle interactions and any final output of  $\mathcal{A}$  from that re-run. After performing a bounded number of such re-runs (controlled below), **MEx** either outputs **fail** or outputs witnesses  $\{x_j\}_{j \in J}$  with each  $x_j$  a valid witness for  $y_j$ .

**Why full-session rewinds per index.** Unlike single-transcript rewinding, concurrency creates cross-session dependencies: the adversary’s future scheduling, commitments, and even which indices become accepting can depend on the exact challenge it received earlier at index  $j$ . Therefore, when targeting index  $j$ , the extractor must re-simulate the adversary’s entire continuation after the  $j$ -th query under fresh randomness and challenges, so as to obtain a second accepting transcript for the same commitment at  $j$  (or determine that the attempt failed and move on). This “first rewind a full session from index  $j$  onward” pattern is crucial to maintain the distributional behavior the adversary expects under concurrency.

**Definition (multi-extractability).** A  $\Sigma$ -protocol  $\Pi$  is multi-extractable if there exists a polynomial-time MEx such that for every polynomially bounded  $B$ , every  $B$ -bounded efficient  $\mathcal{A}$ , and every prescribed  $\theta$  whose inverse is polynomially bounded, there is a negligible  $\eta$  for which: (i)  $\Pr[\text{MEx outputs fail}] \leq \theta + \eta$ ; (ii) the total number of  $\mathcal{A}$  re-runs used by MEx is at most  $\text{poly}(B, 1/\theta)$ ; and (iii) MEx’s overall running time is  $\text{poly}(B, 1/\theta, \lambda)$ . Informally, with probability at least  $1 - \theta - \eta$ , MEx outputs witnesses for every accepting index in the initial run, using at most a polynomial number of full-session rewindings.

**High-level extractor strategy.** - Enumerate accepting indices  $j \in J$  and process them in some order. - For a fixed  $j$ , repeatedly re-run  $\mathcal{A}$  to the  $j$ -th query and supply a fresh challenge. If two accepting transcripts with the same commitment  $t_j$  but distinct challenges are produced, apply special soundness to compute  $x_j$ . If not, cap the number of attempts at  $\text{poly}(1/\theta)$  to keep total failure within the budget; if the cap is hit, record a controlled failure that contributes to the overall  $\leq \theta + \eta$  bound. - Move to the next index in  $J$ ; previously chosen challenges remain fixed because each target is handled by its own full rerun, thus respecting the adversary’s concurrent scheduling and state dependencies.[1][2]

**Intuition on bounds.** Special soundness guarantees that if an index  $j$  yields accepting with non-negligible probability, then two accepting transcripts at  $j$  with different challenges appear within a polynomial number of independent re-challenges. The  $B$ -boundedness ensures only polynomially many indices exist, so the total number of full-session reruns scales polynomially in  $B$  and in  $1/\theta$ . The failure budget  $\theta$  allows the extractor to stop trying on “stubborn” indices without blowing up time, while the negligible slack  $\eta$  covers standard tightness losses.

**Key contrasts vs. single-transcript rewinding.** - Targeted index rewinds are global: after injecting a new challenge at index  $j$ , the extractor must re-simulate the rest of the execution, not only the  $j$ -local transcript, because later commitments and even which sessions become accepting can adapt to that new challenge.[2] - Session-by-session limits: to keep runtime polynomial, the extractor enforces a cap on re-runs per index, trading negligible extraction loss for efficiency within the prescribed  $\theta$ .

**Remark (large challenge space).** When the challenge space is large (so that soundness error per session is small) and the protocol has special soundness, standard analyses yield multi-extraction with a number of re-runs polynomial in  $B$  and  $1/\theta$ , matching the structure above. These analyses rely on the same special-soundness-to-extraction bridge used in the classical case.

# Multi-extraction for $\Sigma$ -protocols under concurrency

**Setup of the game.** Let  $\Pi$  be a  $\Sigma$ -protocol for relation  $\mathcal{R} \subseteq \times$ . A transcript is  $(t, c, z)$  with challenge  $c \in \mathcal{C}$ ,  $|\mathcal{C}| = N$ . Assume *special soundness*: from two accepting  $(t, c, z)$  and  $(t, c', z')$  with  $c \neq c'$  for the same statement  $y$ , one can compute a witness  $x$  for  $y$ .

**$B$ -bounded adversary.** An adversary  $\mathcal{A}$  is  $B$ -bounded if in any execution it issues at most  $B$  verifier-oracle queries overall (across concurrent sessions). Equivalently, one run of  $\mathcal{A}$  is determined by its randomness and a *challenge vector*  $(c_1, \dots, c_B)$ , where  $c_i$  is the response to its  $i$ -th verifier query (unused suffixes can be ignored).

**Initial run and accepting indices.** Run  $\mathcal{A}$  once with uniform challenges to obtain query/response pairs  $((y_1, t_1), c_1), \dots, ((y_r, t_r), c_r)$  with  $r \leq B$ , and  $\mathcal{A}$ 's output  $(J, \{z_j\}_{j \in J})$ , where each  $(t_j, c_j, z_j)$  is accepting for  $y_j$ .

**Goal of the multi-extractor.** Given  $B$  and a prescribed failure budget  $\theta \in (0, 1)$  (with  $1/\theta$  poly-bounded), MEx must output either **fail** or witnesses  $\{x_j\}_{j \in J}$  for all accepting indices, while -  $\Pr[\text{fail}] \leq \theta + \text{negl}(\lambda)$ , - total re-runs of  $\mathcal{A}$  are  $\text{poly}(B, 1/\theta)$ . A convenient sufficient condition used below is  $\theta \geq \frac{2B}{N}$ .

**Full-session rewinding per index.** For each  $j \in J$ , MEx repeatedly re-runs  $\mathcal{A}$  with *rewinding index*  $j$ : the challenge vector becomes

$$(c_1, \dots, c_{j-1}, \tilde{c}_j, \dots, \tilde{c}_B),$$

where  $c_1, \dots, c_{j-1}$  are fixed as in the initial run, and  $\tilde{c}_j, \dots, \tilde{c}_B \leftarrow \mathcal{C}$  are fresh independent challenges. All randomness used by  $\mathcal{A}$  *up to* the  $j$ -th query is set as in the initial run; *after* that point, fresh randomness is used and the entire future execution is re-simulated. This “first rewind session with full rerun” is necessary under concurrency: later commitments, scheduling, and which sessions accept can depend on the new  $j$ -th challenge, so we must simulate the whole continuation to preserve the correct distribution.

**Extraction and stopping rule.** Repeat the above until some  $\tilde{c}_j \neq c_j$  yields an accepting  $(t_j, \tilde{c}_j, \tilde{z}_j)$ ; by special soundness, compute  $x_j$  for  $y_j$ . If extraction at index  $j$  has not succeeded within a cap of at most  $\lceil 2/\theta \rceil$  attempts, record controlled failure and proceed. Process all  $j \in J$  in any order, stopping overall if the total number of re-runs reaches  $\lceil \frac{2B}{\theta} \rceil$ , in which case output **fail**.

**Why the bounds work.** - Large challenge space: per attempt, collision  $c_j = \tilde{c}_j$  occurs with prob.  $1/N$ . - Union bound across at most  $B$  targets and caps of  $\approx 2/\theta$  attempts gives total re-runs  $\leq 2B/\theta$  and failure probability  $\leq \theta$  when  $\theta \geq 2B/N$ . -  $B$ -boundedness keeps concurrency finite so full-session rewinds remain polynomial.

**Key contrast to single-session rewinding.** Single-transcript rewinding changes only the local challenge; under concurrency, changing index  $j$  forces re-simulation of the adversary's *entire future state*, hence full reruns per target index.

## Limitations of rewinding for full CCA: attack setup and blow-up

Below is a self-contained attack description showing why naive rewinding-based extraction fails for full adaptive CCA security of  $\mathcal{E}_{\text{AMEG}}$ .

### 1. Setup: scheme, proofs, and the full CCA game

- **Scheme:** augmented ElGamal  $\mathcal{E}_{\text{AMEG}}$ .
- Each ciphertext is a triple  $(v, e, \pi)$  with

$$v = g^\beta, \quad e = u^\beta \cdot m,$$

and  $\pi$  is a (non-interactive)  $\Sigma$ -proof of knowledge of the randomness  $\beta$ .

- Fiat-Shamir: a random oracle  $H$  (modeled as RO) produces the proof challenge  $c$ .

$$\text{RO\_input} = ((u, v, e), v), \quad c = H(\text{RO\_input}).$$

- **Full CCA game (adaptive):** the adversary can submit decryption queries one by one and wait for each reply before choosing the next query. The reduction / extractor must answer *each* decryption query *immediately*; it cannot wait to see all queries.

### 2. Adversary's high-level strategy (reverse-order dependency)

- The adversary prepares a sequence of  $n$  ciphertexts  $(C_1, \dots, C_n)$  in reverse order so that each  $C_k$  depends on the *future* RO challenges  $(c_{k+1}, \dots, c_n)$ .
- Concretely, for  $k = n, n-1, \dots, 1$ :

- It sets  $m_k, \beta_k$  (and internal values for  $\pi_k$ ) deterministically as a cryptographic hash of  $(c_{k+1}, \dots, c_n)$  and its own secret.
- It then programs/ensures that the RO query for  $C_k$  returns the chosen  $c_k$ .

- Thus  $C_n$  is independent;  $C_{n-1}$  depends on  $c_n$ ;  $C_{n-2}$  depends on  $(c_{n-1}, c_n)$ ; in general

$$C_k = \text{Enc}(m_k(c_{k+1}, \dots, c_n), \beta_k(c_{k+1}, \dots, c_n)).$$

- **Crucial property:** changing any  $c_j$  for  $j > k$  will (with overwhelming probability) change  $C_k$ . This is the adversary's *reverse-dependency trick*.

### 3. What the extractor wants to do: single-ciphertext extraction

- Goal: answer a decryption query  $(v_k^*, e_k^*, \pi_k^*)$  *without* the secret key.
- Standard approach: obtain two accepting transcripts with same first message  $v_k^*$  and different challenges; then by special-soundness extract  $\beta_k^*$  and decrypt

$$m_k^* = e_k^* / u^{\beta_k^*}.$$

- Usual rewinding trick:
  1. Rewind to the snapshot just before the RO return for  $C_k$  (or when  $v_k$  is fixed).
  2. Re-run but have RO output a fresh challenge  $c'_k$ .
  3. If the adversary again produces an accepting transcript with the same commitment  $v_k$  and challenge  $c'_k$ , extract via special soundness.
- This works fine in a *single-session* or non-adaptive setting.

### 4. Why that trick breaks in the adaptive multi-query setting

Consider the extractor trying to answer several adaptive decryption queries in sequence.

- **First decryption query:** adversary sends  $C_1$ .
  - Extractor rewinds around  $C_1$ , obtains a second transcript, extracts  $\beta_1$ , answers the first query. This is fine (and in the AMEG example will succeed with overwhelming probability).
- **Second decryption query:** adversary now sends  $C_2$ .
  - Extractor attempts the same trick: rewind to the snapshot where  $c_2$  was obtained, change  $c_2 \rightarrow c'_2$ , re-run and try to get a second transcript for  $C_2$ .
  - But that snapshot (for  $c_2$ ) is *earlier* than the point where  $C_1$  was produced, because  $C_1$  depends on  $c_2$ .
  - In the re-run with  $c'_2$ , the adversary recomputes  $C_1$  as a function of  $c'_2$  (and unchanged  $c_3, \dots, c_n$ ). So the re-run produces a *different* first ciphertext  $C_1^{**}$  before reaching  $C_2$ .
  - Therefore, the extractor cannot reach the point where it can get the second transcript for  $C_2$  until it has answered / extracted  $C_1^{**}$  in that new run.
- **Recursive problem:**
  - Extracting  $C_1^{**}$  again requires the usual two-transcript procedure.

- To get those two transcripts for  $C_1^{**}$ , the extractor must again rewind to the point where the challenge for  $C_1^{**}$  was given, possibly changing RO replies, which can change other ciphertexts.
- Thus a re-run created to extract  $\beta_2$  forces the extractor to *re-extract* for a fresh first ciphertext in that branch. The previously extracted witness for the original  $C_1$  is useless for  $C_1^{**}$ .
- The same pattern recurses for later queries.

## 5. Quantifying the blow-up: recurrence and closed form

- Let  $f(k)$  be the number of invocations of the special-soundness extractor (i.e., the primitive that needs two accepting transcripts) to fully answer the first  $k$  adaptive decryption queries.

- Base:

$$f(0) = 0.$$

- To respond to the  $k$ -th decryption query:

- You must answer the first  $k - 1$  queries on the main run (cost  $f(k - 1)$ ).
- You then rewind before query  $k$ , run a second independent execution that again reaches the first  $k - 1$  queries (cost another  $f(k - 1)$ ).
- With two runs that reach query  $k$ , you do one more extraction for the  $k$ -th query itself (cost +1).

- Hence the recurrence:

$$f(k) = 2f(k - 1) + 1, \quad f(0) = 0.$$

- Solving gives:

$$f(k) = 2^k - 1.$$

- So answering  $n$  adaptive queries may require up to  $2^n - 1$  primitive extractions: exponential in  $n$ .

## 6. Concrete sketch for $n = 3$ (branch counting)

- Label the main run as  $R_0$ .
- For  $C_1$ :
  - Need two transcripts for  $C_1 \Rightarrow$  cost 1.
  - Thus  $f(1) = 1$ .

- **For  $C_2$ :**
  - Use  $R_0$  as main path.
  - Rewind to pre- $C_2$  and produce a new run  $R_1$  with altered  $c_2$ .
  - $R_1$  yields a new  $C_1^{(1)}$ , so you must extract for this first (cost  $f(1)$  on  $R_1$ ).
  - Then extract for  $C_2$  itself (cost  $+1$ ).
  - So  $f(2) = 2f(1) + 1 = 3$ .
- **For  $C_3$ :**
  - Now you need a second run  $R_2$  with changed  $c_3$ .
  - Each run (main and  $R_2$ ) forces answering the first two queries, costing  $f(2)$  each, plus 1 to extract for  $C_3$ :
$$f(3) = 2f(2) + 1 = 7.$$
- Thus  $f(1) = 1$ ,  $f(2) = 3$ ,  $f(3) = 7$ , matching  $2^k - 1$ .

## 7. Why earlier witnesses cannot be reused

Two natural escape ideas fail:

- **Reuse the old witness for original  $C_1$ :**
  - Impossible: the re-run produces a new ciphertext  $C_1^{**}$  whose witness is a function of changed RO values.
  - The old witness does not satisfy the validity equation for  $C_1^{**}$ , so it does not decrypt it correctly.
- **Force re-runs to keep earlier ciphertexts fixed while changing only later challenges:**
  - In general impossible: earlier ciphertexts were constructed as deterministic functions of the later challenges.
  - You cannot both change a later challenge and keep earlier ciphertexts identical without either:
    - \* programming the RO inconsistently, or
    - \* tampering with the adversary's internal randomness.
  - Both are disallowed in a standard black-box reduction.

Therefore, there is no black-box trick for the extractor to avoid redoing extraction on re-runs; the branching is inherent.

## 8. Conceptual summary

- **Root cause:** the adversary chooses ciphertexts so that earlier ciphertexts are deterministic functions of later RO challenges. Rewinding to alter later challenges therefore changes earlier ciphertexts.
- **Operational problem:** the extractor’s classical tool (rewind to get a second transcript for a target ciphertext) now forces re-generation and fresh extraction for earlier ciphertexts in every re-run branch.
- **Complexity consequence:** the number of extractor invocations follows  $f(k) = 2f(k-1) + 1$ , implying  $f(n) = 2^n - 1$ .
- **Security implication:** this does *not* show  $\mathcal{E}_{\text{AMEG}}$  is insecure; it shows that naive, black-box rewinding + special-soundness extraction cannot efficiently prove full adaptive CCA security for such constructions, because the reduction would need exponential time.

## Subtle Difference Between Parallel-1CCA and Full Adaptive CCA

A key conceptual gap between the parallel-1CCA security model and the full adaptive CCA model lies in the *timing* of the adversary’s random-oracle queries and decryption queries. Although both models involve ciphertexts whose validity is determined by a random oracle, the order in which these oracle queries may occur completely changes the adversary’s capabilities.

**Parallel-1CCA.** In the parallel-1CCA attack game, the adversary must submit all decryption queries *simultaneously*, and receives all responses only after every ciphertext has been submitted. Consequently:

- all ciphertexts  $(v_k, e_k, \pi_k)$  must be fully formed *before* any decryption answer is known;
- every corresponding random-oracle query that defines the challenge  $c_k$  for the proof  $\pi_k$  must also occur before *any* decryption output is revealed.

Hence, the adversary cannot make the witness  $\beta_k$  or the proof randomness depend on unknown future information. In particular, dependencies of the form

$$\beta_1 = F(c_2, c_3, \dots), \quad m_2 = G(m_1, \dots),$$

are impossible, because the values  $c_2, c_3, \dots$  or  $m_1$  simply do not exist at the time the first ciphertext is created. All random-oracle queries “fix” the ciphertext components *once and for all*, and no later information can influence a ciphertext already submitted. This lack of adaptivity is exactly what enables efficient rewinding and multi-extraction.

**Full Adaptive CCA.** In the adaptive CCA attack game, the situation changes dramatically. Here, the adversary is allowed to issue decryption queries one by one, and may wait for the decryption result of query  $k$  before deciding how to construct query  $k + 1$ . Thus, the adversary may proceed as follows:

1. submit ciphertext  $(v_1, e_1, \pi_1)$ ;
2. obtain its decryption result  $m_1$ ;
3. use  $m_1$  to generate fresh random-oracle queries;
4. create the next ciphertext  $(v_2, e_2, \pi_2)$  as a function of  $m_1$  and possibly other future challenges.

Crucially, the adversary may issue random-oracle queries *after* learning  $m_1$ , so that the new challenges  $c_2, c_3, \dots$  depend on earlier decryption outputs. This enables dependency chains such as

$$\beta_2 = H(m_1, c_3, c_4, \text{secret}),$$

which cannot occur in the parallel-1CCA setting. Therefore, while random-oracle queries still “fix” the components of a single ciphertext, they do *not* fix the structure of *future* ciphertexts, because those ciphertexts can be generated only after new information (e.g.,  $m_1$ ) is learned.

**Consequence for Rewinding.** In the parallel-1CCA model, rewinding the adversary with fresh challenges affects all ciphertexts simultaneously, and no ciphertext depends on the decryption of any other. In contrast, in the adaptive CCA model, rewinding at the challenge point for ciphertext  $k$  forces the simulator to re-simulate *all earlier decryption queries* under fresh random-oracle challenges, since those earlier ciphertexts were themselves computed as functions of  $c_k$ . This recursive re-simulation leads to an exponential blow-up, showing why rewinding-based multi-extractors fail in the adaptive CCA setting.

## 8 Fischlin Transform

### Layers of Games and Experiments

We distinguish three conceptual “games” (or experiments) in the NIZK / Fiat–Shamir / online-extractor setting.

**1. Prover–Verifier game (real protocol).** *Purpose:* Defines completeness (and the basic soundness notion) of the protocol.

*Setup:*

- The prover  $P$  knows a witness  $w$  for a statement  $x \in W_k$ .
- The verifier  $V^H$  receives a proof  $\pi = P^H(x, w)$ .

*Game:*

$$V^H(x, \pi) = 1 \quad \text{with overwhelming probability if } x \in W_k \text{ and } \pi \leftarrow P^H(x, w).$$

*Notes:* This is the real NIZK protocol execution, i.e., the “honest world” experiment.

**2. Simulator–Distinguisher game (zero-knowledge).** *Purpose:* Captures zero-knowledge; ensures the proof leaks no information about  $w$ .

*Setup:*

- The simulator is split into two stages  $Z = (Z_0, Z_1)$ .
- $Z_0$  generates an oracle  $H_0$  for the first-stage adversary  $D_0$  and outputs some state  $\delta$ .
- $Z_1$  later sees  $(x, \text{yes/no}, \delta)$  and produces  $(H_1, \pi)$ .

*Game:* A distinguisher  $D = (D_0, D_1)$  is given either

- **Real experiment:**  $\pi$  generated by the real prover using a real random oracle  $H$ , or
- **Simulated experiment:**  $\pi$  generated by the simulator  $Z$  using an oracle  $H_1$ .

The goal of  $D$  is to distinguish between these two cases.

*Notes:* This is the standard ZK game, with  $\delta$  carrying the adversary state from stage 0 to stage 1. Intuitively: can the adversary tell whether it talks to a real prover or a simulator?

**3. Online extractor game.** *Purpose:* Captures the proof-of-knowledge property (soundness with extraction).

*Setup:*

- An adversary  $A^H$  outputs  $(x, \pi)$ , possibly without knowing a witness.
- The random oracle  $H$  records all queries  $Q_H(A)$ .
- The online extractor  $K(x, \pi, Q_H(A))$  sees the oracle transcript and the proof.

*Property:*

$$\Pr[(x, w) \notin W_k \wedge V^H(x, \pi) = 1] \leq \text{negl}(k).$$

Equivalently: any adversary that makes  $V^H$  accept must have implicitly “committed” to a valid witness  $w$ , which  $K$  can recover from  $(x, \pi, Q_H(A))$ .

*Notes:*

- The extractor does not rewind or interact with  $A$ ; it only uses the proof and the online random-oracle transcript.
- The adversary is any PPT algorithm outputting a purported proof.

## Summary as a table

| Game                    | Players                          | Goal                           | Notes                                                                          |
|-------------------------|----------------------------------|--------------------------------|--------------------------------------------------------------------------------|
| Prover–Verifier         | $P, V^H$                         | Completeness / basic soundness | Real protocol between prover and verifier for the scheme.                      |
| Simulator–Distinguisher | $Z = (Z_0, Z_1), D = (D_0, D_1)$ | Zero-knowledge                 | Distinguisher between simulated and real oracle experiments.                   |
| Online extraction       | $A^H, K$                         | Proof-of-knowledge             | Extract a witness from a proof using the oracle transcript, without rewinding. |

## Roles and threat scenarios

We can rephrase the roles as follows:

### 1. Honest world (both honest).

- Scenario:  $P^H$  has a valid witness  $w$ ,  $V^H$  follows the protocol.
- Outcome:  $V^H(x, \pi) = 1$  with overwhelming probability (completeness).
- No simulator or extractor is needed in this case.

### 2. Dishonest prover, honest verifier.

- Scenario: An adversarial prover  $A^H$  tries to convince  $V^H$  to accept for some  $x \notin W_k$ .
- Threat:  $V^H$  might accept an invalid statement, breaking soundness.
- Role of the extractor  $K$ :
  - $K$  observes  $(x, \pi)$  and the random-oracle transcript  $Q_H(A)$ .
  - If  $V^H(x, \pi) = 1$ , then  $K$  outputs a witness  $w$ .
  - Guarantee: If  $K$  fails to find  $w$ , the event  $V^H(x, \pi) = 1$  has only negligible probability.

### 3. Honest prover, dishonest verifier.

- Scenario: A verifier may be malicious and attempt to extract information about  $w$  from  $\pi$  or from the interaction with the random oracle.
- Threat: Leakage of  $w$ , i.e. violation of zero-knowledge.
- Role of the simulator  $Z$ :
  - $Z$  can generate a transcript  $(\pi, H_1)$  that is indistinguishable from a real one, without knowing  $w$ .
  - Even a malicious verifier cannot tell if it is interacting with a real prover or with  $Z$ .

### HVZK vs. special ZK.

- Honest-verifier ZK (HVZK) suffices when the verifier follows the specified randomness and challenge generation.
- Special ZK ensures that zero-knowledge still holds even when challenges are fixed or chosen adversarially in advance (e.g., fixed-challenge experiments).

## High-level takeaway

We can summarize the protections as follows:

| Adversary          | Who is protected? | Mechanism                                                                                                                     |
|--------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Dishonest prover   | Verifier          | Online extractor $K$ shows that any accepting proof yields a witness; cheating without knowledge of $w$ is negligible.        |
| Dishonest verifier | Prover            | Simulator $Z$ shows that transcripts can be generated without $w$ ; thus proofs leak no useful information about the witness. |
| Both honest        | Both              | Completeness: protocol works as intended when all parties follow the specification.                                           |

## 9 Security parameter and negligible functions.

The security parameter is a string  $k$  describing general parameters (e.g.  $k = 1^\lambda$  or  $k = (G, q, g)$  for a group  $G$  of prime order  $q$  and generator  $g$ ). For each  $k$  there is a relation  $W_k \subseteq \{0, 1\}^* \times \{0, 1\}^*$  of statements and witnesses.

A function  $f = f(k)$  is *negligible* if for every polynomial  $p(\cdot)$  there exists  $k_0$  such that for all  $k \geq k_0$  we have  $|f(k)| \leq 1/p(k)$ . If  $f - g$  is negligible we write  $f \approx g$ . A function  $f$  is *overwhelming* if  $1 - f$  is negligible.

**Random oracle model.** In the random oracle model, all parties have oracle access to a random function  $H$  whose domain and range depend on  $k$ . The relation  $W_k$  does not depend on  $H$  (to avoid self-reference). In some definitions an algorithm may output a description of a pseudorandom function to be used as  $H$ .

### 9.1 Fiat Shamir

[Fiat–Shamir proof of knowledge] A Fiat–Shamir proof of knowledge for a relation  $W = (W_k)_k$  with  $k$ -bit challenges consists of a pair of PPT algorithms

$$P_{\text{FS}} = (P_0, P_1), \quad V_{\text{FS}} = (V_0, V_1),$$

with the following syntax:

- $P_0(x, w)$  outputs a commitment **com**,
- $V_0(x, \text{com})$  outputs a challenge  $\text{ch} \in \{0, 1\}^k$ ,
- $P_1(x, w, \text{com}, \text{ch})$  outputs a response **resp**,
- $V_1(x, \text{com}, \text{ch}, \text{resp}) \in \{0, 1\}$  decides accept/reject.

We write

$$(\text{com}, \text{ch}, \text{resp}) \leftarrow P_{\text{FS}}(x, w; V_0(x, \cdot))$$

for the interactive execution between  $P_{\text{FS}}$  and  $V_{\text{FS}}$ . The following properties must hold (all probabilities over the internal randomness of the algorithms):

**Completeness.** For all  $k$ , all  $(x, w) \in W_k$ , if  $(\text{com}, \text{ch}, \text{resp})$  is generated by an honest interaction on input  $(x, w)$ , then

$$\Pr [V_1(x, \text{com}, \text{ch}, \text{resp}) = 1] = 1.$$

**Commitment entropy.** For all  $k$  and all  $(x, w) \in W_k$ , the commitment  $\text{com} \leftarrow P_0(x, w)$  has superlogarithmic min-entropy as a function of  $k$ .

**Public coin.** For all  $k$ , all  $(x, w) \in W_k$  and all  $\text{com}$  in the support of  $P_0(x, w)$ , the challenge  $\text{ch} \leftarrow V_0(x, \text{com})$  is uniform over  $\{0, 1\}^k$ .

**Unique responses (quasi-uniqueness).** For every PPT algorithm  $A$ , consider  $(x, \text{com}, \text{ch}, \text{resp}, \text{resp}') \leftarrow A(k)$ . Then

$$\Pr \left[ V_1(x, \text{com}, \text{ch}, \text{resp}) = 1 \wedge V_1(x, \text{com}, \text{ch}, \text{resp}') = 1 \wedge \text{resp} \neq \text{resp}' \right] \text{ is negligible in } k.$$

**Special soundness.** There exists a PPT extractor  $K_{\text{FS}}$  such that for all  $k$ , all  $x$ , and all pairs

$$(\text{com}, \text{ch}, \text{resp}), (\text{com}, \text{ch}', \text{resp}')$$

with  $\text{ch} \neq \text{ch}'$  and

$$V_1(x, \text{com}, \text{ch}, \text{resp}) = V_1(x, \text{com}, \text{ch}', \text{resp}') = 1,$$

the output  $w \leftarrow K_{\text{FS}}(x, \text{com}, \text{ch}, \text{resp}, \text{ch}', \text{resp}')$  satisfies  $(x, w) \in W_k$  except with negligible probability.

**Honest-verifier zero knowledge.** There exists a PPT simulator  $Z_{\text{FS}}$  such that for any PPT distinguisher  $D = (D_0, D_1)$ , the following two experiments are computationally indistinguishable:

- *Real:*  $x, w \leftarrow D_0(k)$ . If  $(x, w) \in W_k$  run an honest execution of  $P_{\text{FS}}(x, w)$  with  $V_0(x, \cdot)$  to obtain  $(\text{com}, \text{ch}, \text{resp})$ ; otherwise set  $(\text{com}, \text{ch}, \text{resp})$  to some fixed invalid triple. Output  $D_1(\text{com}, \text{ch}, \text{resp}, \cdot)$ .
- *Simulated:*  $x, w \leftarrow D_0(k)$ . If  $(x, w) \in W_k$  set  $(\text{com}, \text{ch}, \text{resp}) \leftarrow Z_{\text{FS}}(x, \text{yes})$ , otherwise  $(\text{com}, \text{ch}, \text{resp}) \leftarrow Z_{\text{FS}}(x, \text{no})$ . Output  $D_1(\text{com}, \text{ch}, \text{resp}, \cdot)$ .

**[Honest-verifier zero knowledge]** Let  $W = (W_k)_k$  be a relation. A 3-move public-coin proof system  $(P_{\text{FS}}, V_{\text{FS}})$  for  $W$  is *honest-verifier zero knowledge* if there exists a PPT simulator  $Z_{\text{FS}}$  such that for every two-stage PPT distinguisher  $D = (D_0, D_1)$  the following two experiments are computationally indistinguishable.

**Real experiment**  $\text{Real}_D^{P_{\text{FS}}, V_{\text{FS}}}(k)$ .

()

1. Run  $(x, w, \delta) \leftarrow D_0(k)$ .
2. If  $(x, w) \in W_k$  then run an honest execution of  $P_{\text{FS}}$  with  $V_{\text{FS}}$  on input  $(x, w)$ :

$$\begin{aligned} \text{com} &\leftarrow P_0(x, w), \\ \text{ch} &\leftarrow V_0(x, \text{com}), \\ \text{resp} &\leftarrow P_1(x, w, \text{com}, \text{ch}), \end{aligned}$$

and set  $(\text{com}, \text{ch}, \text{resp})$  to this transcript. Otherwise (if  $(x, w) \notin W_k$ ), set  $(\text{com}, \text{ch}, \text{resp})$  to some fixed dummy triple.

3. Output the bit

$$b_{\text{real}} \leftarrow D_1(\text{com}, \text{ch}, \text{resp}, \delta).$$

**Simulated experiment**  $\text{Sim}_D^{Z_{\text{FS}}}(k)$ .

1. Run  $(x, w, \delta) \leftarrow D_0(k)$ .

2. If  $(x, w) \in W_k$  then compute  $(\text{com}, \text{ch}, \text{resp}) \leftarrow Z_{\text{FS}}(x, \text{yes})$ . Otherwise compute  $(\text{com}, \text{ch}, \text{resp}) \leftarrow Z_{\text{FS}}(x, \text{no})$ . (In both cases the simulator only receives  $x$  and the bit **yes/no**, not the witness  $w$ .)

3. Output the bit

$$b_{\text{sim}} \leftarrow D_1(\text{com}, \text{ch}, \text{resp}, \delta).$$

The protocol is honest-verifier zero knowledge if for every PPT  $D$  the distributions  $\{b_{\text{real}}\}_k$  and  $\{b_{\text{sim}}\}_k$  are computationally indistinguishable as functions of  $k$ .

## Special zero knowledge

Let  $W = (W_k)_k$  be a relation and let  $(P_{\text{FS}}, V_{\text{FS}})$  be a 3-move public-coin proof system as before, with challenge length  $\ell(k)$ . For any fixed challenge  $\text{ch}$ , define the verifier  $V_0^{\text{ch}}$  that on input  $(x, \text{com})$  simply outputs  $\text{ch}$  (i.e.,  $V_0^{\text{ch}}(x, \text{com}) = \text{ch}$ ).

The protocol is *special zero knowledge* if there exists a PPT simulator  $Z_{\text{sp}}$  (the special ZK simulator) such that for every two-stage PPT distinguisher  $D = (D_0, D_1)$  and every security parameter  $k$  the following two experiments are computationally indistinguishable.

**Real experiment**  $\text{Real}_D^{\text{sp}}(k)$ .

1. Run  $(x, w, \text{ch}, \delta) \leftarrow D_0(k)$ .

2. If  $(x, w) \in W_k$  then run an honest execution of the prover against the verifier that always uses challenge  $\text{ch}$ :

$$\begin{aligned} \text{com} &\leftarrow P_0(x, w), \\ \text{resp} &\leftarrow P_1(x, w, \text{com}, \text{ch}), \end{aligned}$$

and set  $(\text{com}, \text{ch}, \text{resp})$  to this transcript. Otherwise (if  $(x, w) \notin W_k$ ) set  $(\text{com}, \text{ch}, \text{resp}) := \perp$ .

3. Output

$$b_{\text{real}} \leftarrow D_1(\text{com}, \text{ch}, \text{resp}, \delta).$$

**Simulated experiment**  $\text{Sim}_D^{\text{sp}}(k)$ .

1. Run  $(x, w, \text{ch}, \delta) \leftarrow D_0(k)$ .
2. If  $(x, w) \in W_k$  then compute  $(\text{com}, \text{ch}, \text{resp}) \leftarrow Z_{\text{sp}}(x, \text{ch}, \text{YES})$ . Otherwise compute  $(\text{com}, \text{ch}, \text{resp}) \leftarrow Z_{\text{sp}}(x, \text{ch}, \text{NO})$ . (The simulator sees the statement  $x$ , the externally chosen challenge  $\text{ch}$ , and a bit indicating whether a witness exists, but does not get  $w$ .)
3. Output

$$b_{\text{sim}} \leftarrow D_1(\text{com}, \text{ch}, \text{resp}, \delta).$$

We say  $(P_{\text{FS}}, V_{\text{FS}})$  is special zero knowledge if for every PPT  $D$  the distributions  $\{b_{\text{real}}\}_k$  and  $\{b_{\text{sim}}\}_k$  are computationally indistinguishable.

**Difference from plain HVZK (previous definition):**

- In HVZK, the challenge  $\text{ch}$  is generated honestly by  $V_0(x, \text{com})$ ; the simulator must produce a whole transcript with a **fresh** random challenge.
- In special ZK, the distinguisher  $D_0$  **fixes**  $\text{ch}$  up front, and both the real execution and the simulator must use that exact challenge. The simulator therefore takes  $(x, \text{ch}, \text{YES/NO})$  as input and must fabricate a transcript consistent with that specific challenge.

This stronger ability (“simulate for any externally chosen  $\text{ch}$ ”) is what they later need to build the ROM simulator that first chooses all hash outputs and then stitches them into valid-looking FS transcripts.

## 9.2 NIZK proof of knowledge with online extractor

Let  $W = (W_k)_k$  be a relation. A non-interactive zero-knowledge proof of knowledge for  $W$  with online extractor in the random oracle model consists of two PPT algorithms  $P$  and  $V$  with oracle access to a random oracle  $H$ , such that:

- The prover  $P^H(x, w)$  outputs a proof  $\pi$  for statement  $x$ .
- The verifier  $V^H(x, \pi) \in \{0, 1\}$  decides accept/reject.

The following properties must hold:

**Completeness.** For all  $k$ , all  $(x, w) \in W_k$ , and all random oracles  $H$ ,

$$\Pr [V^H(x, P^H(x, w)) = 1] = 1.$$

**Zero knowledge in ROM.** There exists a two-stage PPT simulator  $Z = (Z_0, Z_1)$  such that for every two-stage PPT distinguisher  $D = (D_0, D_1)$  the following two experiments are computationally indistinguishable:

- *Real experiment.* Let  $H$  be a random oracle. Run  $(x, w, \sigma) \leftarrow D_0^H(k)$ . If  $(x, w) \in W_k$  set  $\pi \leftarrow P^H(x, w)$ , else set  $\pi$  to some fixed invalid value. Output  $D_1^H(\pi, \sigma)$ .
- *Simulated experiment.* Simulator  $Z_0$  outputs an initial oracle  $H_0$  and simulator state  $\tau$ . Run  $(x, w, \sigma) \leftarrow D_0^{H_0}(k)$ . Then  $Z_1$  on input  $(k, x, \tau, \mathbf{b})$ , where  $\mathbf{b} = \text{yes}$  if  $(x, w) \in W_k$  and  $\mathbf{b} = \text{no}$  otherwise, outputs a completed oracle  $H_1$  (extending  $H_0$  consistently) and a proof  $\pi$ . Output  $D_1^{H_1}(\pi, \sigma)$ .

**Zero knowledge in the random oracle model.** A non-interactive proof system  $(P, V)$  for a relation  $W = (W_k)_k$  in the random oracle model is *zero knowledge* if there exists a two-stage PPT simulator  $Z = (Z_0, Z_1)$  such that for every two-stage PPT distinguisher  $D = (D_0, D_1)$  the following two experiments are computationally indistinguishable.

**Real experiment**  $\text{Real}^{P,V,D}(k)$ .

1. Sample a random oracle  $H$  (a truly random function of the appropriate domain and range).
2. Run the first stage of the distinguisher, with oracle access to  $H$ :

$$(x, w, \sigma) \leftarrow D_0^H(k).$$

Here  $D_0$  is allowed to make arbitrary (polynomially many) queries to  $H$  while choosing a statement  $x$ , some string  $w$  (which may or may not be a true witness), and side information  $\sigma$  that it passes to  $D_1$ .

3. If  $(x, w) \in W_k$ , run the honest prover with oracle  $H$  to produce a proof:

$$\pi \leftarrow P^H(x, w).$$

Otherwise (if  $(x, w) \notin W_k$ ) set  $\pi$  to some fixed dummy value (for example, a fixed non-accepting string of the right length).

4. Run the second stage of the distinguisher, again with access to the *same* oracle  $H$ :

$$b_{\text{real}} \leftarrow D_1^H(\pi, \sigma).$$

Output  $b_{\text{real}}$ .

**Simulated experiment**  $\text{Sim}^{Z,D}(k)$ .

1. The first-stage simulator  $Z_0$  chooses an initial description of an oracle and its own internal state:

$$(H_0, \tau) \leftarrow Z_0(k).$$

Think of  $H_0$  as a “partially programmed” random oracle: it specifies answers for some inputs and leaves the rest undefined;  $Z_0$  also keeps state  $\tau$  for later.

2. Run the first stage of the distinguisher with oracle  $H_0$ :

$$(x, w, \sigma) \leftarrow D_0^{H_0}(k).$$

During this step,  $D_0$  can freely query  $H_0$  to set up its view and adaptively choose  $(x, w, \sigma)$ , exactly as in the real experiment, except that the answers now come from  $H_0$  chosen by the simulator.

3. Define a bit

$$\mathbf{b} := \begin{cases} \text{yes} & \text{if } (x, w) \in W_k, \\ \text{no} & \text{otherwise.} \end{cases}$$

Run the second-stage simulator on input  $(k, x, \tau, \mathbf{b})$ :

$$(H_1, \pi) \leftarrow Z_1(k, x, \tau, \mathbf{b}).$$

Here  $Z_1$  must:

- extend  $H_0$  to a full oracle  $H_1$  by defining answers on any previously undefined inputs;
- keep  $H_1$  *consistent* with all answers that  $D_0$  already saw from  $H_0$ ;
- output a simulated proof  $\pi$  for statement  $x$  (using only  $x$  and the bit  $\mathbf{b}$ , not the witness  $w$ ).

4. Run the second stage of the distinguisher with oracle  $H_1$ :

$$b_{\text{sim}} \leftarrow D_1^{H_1}(\pi, \sigma).$$

Output  $b_{\text{sim}}$ .

Zero knowledge in the ROM requires that the distributions  $\{b_{\text{real}}\}_k$  and  $\{b_{\text{sim}}\}_k$  are computationally indistinguishable as functions of  $k$ .

**Online extractor.** There exists a PPT extractor  $K$  such that for every PPT prover algorithm  $A$  the following holds. Let  $H$  be a random oracle, and run  $(x, \pi) \leftarrow A^H(k)$ , where  $A$  makes oracle queries to  $H$ . Let  $Q_H^A$  be the list of all query/answer pairs of  $A$  to  $H$ . Define  $w \leftarrow K(x, \pi, Q_H^A)$ . Then, as a function of  $k$ ,

$$\Pr [V^H(x, \pi) = 1 \wedge (x, w) \notin W_k] \text{ is negligible.}$$

The extractor in general sense is defined for a malicious prover who try to convince the verifier the proof for false statements in the above definition  $A$  is playing the role of the such a prover who can query the RO and outputs a pair  $(x, \pi)$  to the verifier.

## 10 Fischlin Transform generic construction

**Parameters and setup** All parameters are functions of the security parameter  $k$ .

- Relation  $(W = \{W_k\}_k)$ .
- An interactive Fiat–Shamir (FS)  $\Sigma$ -protocol  $(P_{\text{FS}}, V_{\text{FS}})$  for  $W$  is given. Its challenge length is

$$\ell_{\text{FS}} = \ell(k),$$

satisfying  $\ell_{\text{FS}} = O(\log k)$ .

- Random oracle  $H : \{0, 1\}^* \rightarrow \{0, 1\}^b$ , assumed to return  $b$ -bit values.

Concrete parameters (all functions of  $k$ ):

$$\begin{aligned} b = b(k) & \quad \text{number of hash bits, output length,} \\ t = t(k) & \quad \text{number of challenge bits searched per repetition,} \\ r = r(k) & \quad \text{number of independent repetitions,} \\ S = S(k) & \quad \text{verifier's threshold for the hash sum.} \end{aligned}$$

These satisfy:

$$\begin{aligned} b, r, t &= O(\log k), & b \leq t \leq \ell_{\text{FS}}, & & br = \omega(\log k), \\ 2^{t-b} &= \omega(\log k), & S &= O(r). \end{aligned}$$

**Notation** For repetition index  $i \in \{1, \dots, r\}$ :

$$\begin{aligned} \text{com}_i & \quad \text{commitment produced in repetition } i, \\ \text{ch}_i & \quad \text{chosen challenge in } \{0, \dots, 2^t - 1\}, \\ \text{resp}_i & \quad \text{response for } (\text{com}_i, \text{ch}_i), \\ s_i & \quad := H(x, \text{com}, i, \text{ch}_i, \text{resp}_i) \in \{0, 1\}^b, \end{aligned}$$

where  $\text{com} = (\text{com}_1, \dots, \text{com}_r)$  and  $s_i$  is viewed as an integer in  $[0, 2^b - 1]$ .

The final proof is the tuple

$$\pi = ((\text{com}_i, \text{ch}_i, \text{resp}_i))_{i=1}^r.$$

**Prover algorithm**  $P^H(x, w)$  Input: statement  $x$ , witness  $w$ , and oracle access to  $H$ .

1. Run  $P_{\text{FS}}(x, w)$  independently  $r$  times up to the commitment stage. Obtain  $\text{com}_1, \dots, \text{com}_r$  and set  $\text{com} = (\text{com}_1, \dots, \text{com}_r)$ .
2. For each  $i \in \{1, \dots, r\}$ :

- (a) For each candidate  $\text{ch} \in \{0, 1, \dots, 2^t - 1\}$ :
  - i. Run (or rewind) the FS prover to compute  $\text{resp}(\text{ch})$  for  $(\text{com}_i, \text{ch})$ .
  - ii. Compute  $h := H(x, \mathbf{com}, i, \text{ch}, \text{resp}(\text{ch})) \in \{0, 1\}^b$  and interpret  $h$  as an integer in  $[0, 2^b - 1]$ .

(b) Selection rule:

- If  $\exists \text{ch}$  with  $h = 0$ , choose the first such  $\text{ch}$  and set

$$\text{ch}_i := \text{ch}, \quad \text{resp}_i := \text{resp}(\text{ch}), \quad s_i := 0.$$

- Otherwise, choose  $\text{ch}$  minimizing  $h$  and set

$$\text{ch}_i := \text{ch}, \quad \text{resp}_i := \text{resp}(\text{ch}), \quad s_i := h.$$

3. Output the proof

$$\pi = ((\text{com}_i, \text{ch}_i, \text{resp}_i))_{i=1}^r.$$

Worst-case cost:

$O(r \cdot 2^t)$  FS response computations and the same number of hash evaluations.

**Verifier algorithm**  $V^H(x, \pi)$  Input: statement  $x$ , proof  $\pi = ((\text{com}_i, \text{ch}_i, \text{resp}_i))_{i=1}^r$ , and oracle access to  $H$ .

1. For each  $i = 1, \dots, r$ , verify:

$$V_{1,\text{FS}}(x, \text{com}_i, \text{ch}_i, \text{resp}_i) = 1.$$

If any fails, reject.

2. For each  $i$ , compute

$$s_i := H(x, \mathbf{com}, i, \text{ch}_i, \text{resp}_i) \in [0, 2^b - 1].$$

3. Let

$$T := \sum_{i=1}^r s_i.$$

4. Accept iff  $T \leq S$ ; otherwise reject.

Verifier cost:  $r$  FS verifications and  $r$  hash evaluations.

**Acceptance conditions** The verifier accepts  $(x, \pi)$  iff both hold:

$$\forall i : V_{1,\text{FS}}(x, \text{com}_i, \text{ch}_i, \text{resp}_i) = 1,$$

$$\text{and } \sum_{i=1}^r H(x, \mathbf{com}, i, \text{ch}_i, \text{resp}_i) \leq S.$$

## Intuition and remarks

- **Repetitions.** Each repetition is an independent sub-proof; repeating  $r = O(\log k)$  times achieves negligible soundness error.
- **Search over  $2^t$  challenges.** The honest prover can compute responses for all challenges, thus can search for “good” hashes efficiently.
- **Good hashes.** The event  $H(\cdot) = 0_b$  occurs with probability  $2^{-b}$ ; the prover finds one with high probability across  $2^t$  trials. If none found, the minimum  $h$  is chosen for completeness.
- **Sum threshold.** The check  $T \leq S$  bounds the total “smallness” of hashes, limiting the chance a cheating prover can pass without a witness.
- **Verifier efficiency.** The verifier only performs  $O(r)$  work regardless of the prover’s internal search.
- **Extractor intuition.** In the random oracle model, an extractor monitors oracle queries, reconstructs multiple valid  $(\text{com}, \text{ch}, \text{resp})$  tuples, and applies special soundness to extract a witness when the verifier would accept.

**Example parameters** A concrete toy setting:

$$b = 9, \quad t = 12, \quad r = 10, \quad S = 10.$$

Then each “good hash” (all-zero) occurs with probability  $2^{-9}$  per trial; with  $2^{12}$  trials per repetition, an honest prover almost always succeeds, whereas a cheating prover’s success probability across ten repetitions becomes negligible.

## Complexity summary

$$\begin{aligned} \text{Prover time:} & \quad O(r \cdot 2^t \cdot T_{\text{FS}}), \\ \text{Verifier time:} & \quad O(r \cdot (T_{\text{VFS}} + T_H)), \\ \text{Proof size:} & \quad r \text{ tuples } (\text{com}_i, \text{ch}_i, \text{resp}_i). \end{aligned}$$

## 11 Proof of above construction

In this section we will prove that above construction is a NIZK with online simulator security in random oracle model. Recall that the first property is **Completeness**.

## Completeness

If the prover is honest then, The following properties must hold:

**Completeness.** For all  $k$ , all  $(x, w) \in W_k$ , and all random oracles  $H$ ,

$$\Pr [V^H(x, P^H(x, w)) = 1] = 1.$$

Since the prover is honest the FS verification will always holds so the first part of the verification is over. The major part is the second condition which states that the sum of  $r$ -hashes defined  $s_i = H(x, com, i, ch_i, resp_i)$  is less than the value  $S$ .

$$\sum_{i=1}^r s_i \leq S$$

So it is enough to argue that this will happen with negligible probability.

$S = 0, 1, \dots, S$

$$\text{Prob}[s_i \leq S] \leq r \cdot \frac{(S+1)^{2^t}}{2^b}$$

because in each round prover tries  $2^t$  challenges to get a random hash value of atmost  $S$  with above probability and all hash values are independent with  $H$  is random oracle.

$$\begin{aligned} \text{Prob}[\exists i : s_i > S] &\leq r \cdot \left(1 - \frac{(S+1)^{2^t}}{2^b}\right)^{2^t} \\ &\leq r \cdot e^{-(S+1)2^{t-b}} \end{aligned}$$

Hence the probability of having larger value than  $S$  in each execution is negligible because  $r$  is logarithmic and  $2^{t-b}$  is superlogarithmic. Hence the sum of all values will exceeds  $rS$  is negligible.

Now the next part is although the sum is bounded by  $rS$  this is not enough we need to argue it is bounded by  $S$  with high probability.

Define the sum as

$$T = \sum_{i=1}^r s_i$$

then  $T = S+1, S+2, \dots, rS$  is the failing condition for honest prover.

For any such  $T$  there are at most  $\binom{T+r-1}{r-1}$  ways to partition  $T$  into  $r$  non-negative integers  $s_1, s_2, \dots, s_r$

This is upper bounded by

$$\binom{T+r-1}{r-1} \leq \left( \frac{e(rS+r-1)}{r-1} \right)^{r-1} \leq (e(2S+1))^{r-1} \leq e^{r \cdot \ln(e(2S+1))}$$

On the other hand the probability of obtaining such a sum for a given partition,  $s_1 = S_1, s_2 = S_2, \dots, s_r = S_r$  is atmost

$$\begin{aligned} \prod_{i=1}^r \text{Prob}[S_i = s_i] &\leq \prod_{i=1}^r \text{Prob}[s_i \geq S_i] \leq \left(1 - \frac{S_i}{2^b}\right)^{2^t} \\ &\leq \prod_{i=1}^r e^{-S_i 2^{t-b}} = e^{-(\sum S_i) 2^{t-b}} = e^{-T 2^{t-b}} \leq e^{-(S+1) 2^{t-b}} \end{aligned}$$

By choice of parameters the probability of getting a sum  $T$  with  $S < T < rS$  is therefore limited by  $\exp(r \ln(e(2S+1)) - (S+1) 2^{t-b})$ . since  $\ln(2S+1) \leq 2S+1$ ,  $r = O(\log k)$  and  $2^{t-b} = w(\log k)$  this is negligible.

## Completeness (failure probability bound)

We show that an honest prover is accepted except with negligible probability. Fix the security parameter  $k$ . Let the protocol parameters be as in Construction 1: the random oracle  $H$  outputs  $b$  bits, the prover makes up to  $2^t$  trials per repetition, and the protocol has  $r$  independent repetitions. For repetition index  $i$  let the prover's final (selected) hash value be

$$h_i := H(x, \mathbf{com}, i, \text{ch}_i, \text{resp}_i) \in \{0, 1, \dots, 2^b - 1\},$$

interpreted as an integer, where  $\mathbf{com} = (\text{com}_1, \dots, \text{com}_r)$ . The verifier accepts only when the underlying FS-verification holds for each repetition and the sum

$$T := \sum_{i=1}^r h_i$$

satisfies  $T \leq S$ . Completeness (honest prover) requires that the honest prover produces such  $h_1, \dots, h_r$  except with negligible probability. We prove the following bound on the failure event  $T > S$ .

[Honest-prover failure is negligible] Under the parameter constraints

$$b, r, t = O(\log k), \quad br = \omega(\log k), \quad 2^{t-b} = \omega(\log k), \quad (1)$$

the probability that an honest prover (who knows a witness) outputs a proof with  $T > S$  is negligible in  $k$ . More precisely,

$$\Pr [T > S] \leq r \cdot \exp \left( - (S + 1) 2^{t-b} \right) + \exp(r \ln(e(2S + 1)) - (S + 1) 2^{t-b}),$$

which is negligible in  $k$  for parameter choices satisfying (1).

*Proof.* The honest prover acts as follows in each repetition  $i$ : it fixes a commitment  $\text{com}_i$ , and then tries up to  $2^t$  challenges  $c \in \{0, 1, \dots, 2^t - 1\}$ . For each trial it computes the corresponding valid FS response  $\text{resp}(c)$  (since the prover knows the witness) and evaluates the random oracle on the tuple  $(x, \text{com}_i, i, c, \text{resp}(c))$ , obtaining an independent uniformly random  $b$ -bit string which we view as an integer in  $[0, 2^b - 1]$ . The prover selects the first trial that achieves the desired property (e.g. minimal hash or  $b$  zero LSBs) and outputs the corresponding  $(\text{com}_i, c, \text{resp})$ . Thus for each repetition the prover effectively obtains  $\leq 2^t$  i.i.d. uniform samples from  $\{0, \dots, 2^b - 1\}$  and selects the minimum (or the first sample satisfying the test). We now bound the probability that the final sum  $T$  exceeds  $S$ .

**Step 1: per-repetition tail bound.** Fix a repetition  $i$ . For a single random oracle sample  $X \in \{0, \dots, 2^b - 1\}$  we have  $\Pr[X \leq S] = (S + 1)/2^b$ . The probability that *none* of the  $2^t$  independent trials satisfies  $X \leq S$  is

$$\left(1 - \frac{S + 1}{2^b}\right)^{2^t}.$$

Using the standard inequality  $1 - u \leq e^{-u}$  for  $u \in \mathbb{R}$  we get

$$\left(1 - \frac{S+1}{2^b}\right)^{2^t} \leq \exp(-(S+1)2^{t-b}).$$

Thus the probability that the prover fails to obtain a trial with value  $\leq S$  in a given repetition is at most  $\exp(-(S+1)2^{t-b})$ .

**Step 2: union bound for the event “some  $h_i > S$ ”.** By the union bound over the  $r$  repetitions,

$$\Pr[\exists i : h_i > S] \leq r \cdot \exp(-(S+1)2^{t-b}).$$

Since  $r = O(\log k)$  and  $2^{t-b} = \omega(\log k)$  by (1), the right-hand side is negligible. Therefore with overwhelming probability each selected  $h_i$  satisfies  $h_i \leq S$ . From now on we condition on the complementary event (that no single  $h_i$  exceeds  $S$ ) when bounding the probability that the total sum  $T$  nevertheless exceeds  $S$ .

**Step 3: partitioning the bad total  $T$ .** Suppose  $T > S$ . Then necessarily  $T$  is an integer with  $S+1 \leq T \leq rS$ . Fix such a value  $T$ . The number of nonnegative integer tuples  $(h_1, \dots, h_r)$  with  $\sum_{i=1}^r h_i = T$  is the standard stars-and-bars coefficient

$$N(T, r) = \binom{T+r-1}{r-1}.$$

We will upper bound  $N(T, r)$  and then multiply by an upper bound on the probability that a *specific* tuple occurs.

A standard binomial upper bound (from  $\binom{n}{k} \leq (en/k)^k$ ) yields

$$N(T, r) = \binom{T+r-1}{r-1} \leq \left(\frac{e(T+r-1)}{r-1}\right)^{r-1}.$$

For our range  $T \leq rS$  we therefore have

$$N(T, r) \leq \left(\frac{e(rS+r-1)}{r-1}\right)^{r-1} \leq (e(2S+1))^{r-1},$$

where the last inequality uses  $r \geq 2$  and  $rS+r-1 \leq (2S+1)(r-1)$  for  $S \geq 0$ .

**Step 4: probability of a fixed partition.** Fix a particular partition  $(s_1, \dots, s_r)$  with  $\sum_i s_i = T$ . We bound the probability that the honest prover’s selected hashes equal these values. For each repetition  $i$ , the prover obtains up to  $2^t$  independent samples from the uniform distribution on  $\{0, \dots, 2^b-1\}$ . An upper bound on the probability that the minimum sample (or the chosen sample) is *at least*  $s_i$  is

$$\Pr[\text{all trials in repetition } i \text{ are } \geq s_i] = \left(1 - \frac{s_i}{2^b}\right)^{2^t}.$$

(Here we use that among the  $2^t$  uniform samples none falls in  $\{0, \dots, s_i - 1\}$ .) Hence for the fixed partition we obtain

$$\Pr[(h_1, \dots, h_r) = (s_1, \dots, s_r)] \leq \prod_{i=1}^r \left(1 - \frac{s_i}{2^b}\right)^{2^t}.$$

Using  $1 - u \leq e^{-u}$  again gives

$$\prod_{i=1}^r \left(1 - \frac{s_i}{2^b}\right)^{2^t} \leq \exp\left(-2^t \sum_{i=1}^r \frac{s_i}{2^b}\right) = \exp(-T2^{t-b}).$$

**Step 5: sum over partitions and final bound.** Combining the combinatorial count with the per-partition probability bound, the probability that there exists a partition realizing a total sum  $T$  is at most

$$N(T, r) \cdot \exp(-T2^{t-b}) \leq (e(2S + 1))^{r-1} \cdot \exp(-T2^{t-b}).$$

Summing over all possible bad totals  $T = S + 1, \dots, rS$  yields the bound

$$\Pr[S < T \leq rS] \leq \sum_{T=S+1}^{rS} (e(2S + 1))^{r-1} e^{-T2^{t-b}} \leq rS \cdot (e(2S + 1))^{r-1} \cdot e^{-(S+1)2^{t-b}}.$$

Absorbing polynomial-in- $r, S$  factors into the exponent (and recalling  $r = O(\log k)$ ) we obtain the simpler bound stated in the lemma:

$$\Pr[S < T \leq rS] \leq \exp(r \ln(e(2S + 1)) - (S + 1)2^{t-b}).$$

**Step 6: negligibility.** By the parameter constraints (1) we have  $r \ln(e(2S + 1)) = O(\log k)$  while  $(S + 1)2^{t-b} = \omega(\log k)$ . Hence the exponent above is  $-\omega(\log k)$ , i.e. the right-hand side is negligible in  $k$ . Combining this with the union-bound term from Step 2 finishes the proof: the overall probability that  $T > S$  (either because some  $h_i > S$  or because  $S < T \leq rS$ ) is negligible.  $\square$

## 11.1 Zero knowledge

In this section we will prove the special zero knowledge property define above The Zero knowledge simulator as per defined  $Z = (Z_0, Z_1)$  in the first stage simply lets  $H_0$  be a pseudo-random blackbox oracle. For the second stage  $Z_1$  defines  $H_1$  to be consistent with  $H_0$  on previous queries. For any other query to  $H_1$  simulator  $Z_1$  on input  $x$ , first sample  $2^t$  random b-bit strings for each  $i$  and assigns them to the t-bit challenges  $ch_i \in \{0, 1\}^t$ . Let  $\tau_i : \{0, 1\}^t \rightarrow \{0, 1\}^b$  describes this assignment. Let  $ch_i$  be the first one in lexicographic ordering the minimum over all these  $2^t$  values that is  $(\tau(ch_i))$  is minimum).  $Z_1$  next runs the special zero knowledge simulator  $Z_{FS}$  of the underlying Fiat shamir proof  $r$  times on  $x$

and each  $ch_i$  to obtain  $r$  tuples  $(com_i, ch_i, resp_i)$ . It then defines the hash function  $H_1$  for any query  $(x, com, i, ch_i^*, resp_i^*)$  with  $V_1(x, com, i, ch_i^*, resp_i^*) = 1$  to be the value  $\tau(ch_i^*)$ . All other values  $H_1$  are chosen randomly this includes RO queries other than the above useful one. The simulator outputs  $\pi = (com_i, ch_i, resp_i)_{i=1, \dots, r}$  as the proof.

Assume for the moment that the tuples  $(com_i, ch_i, resp_i)$  are generated by genuine runs between  $P_{FS}(x, w)$  and  $V_{FS}^{ch_i}(x, ch_i)$ , instead of being sampled through  $Z_{FS}(x, ch_i)$ . Recall that  $V_{FS}^{ch}(x, ch)$  is the special verifier which merely copies  $ch$ .

We only have to show that the simulation of  $Z = (Z_0, Z_1)$  on these tuples, called *hybrid simulation*, cannot be distinguished from the genuine behavior between  $P$  and  $V$  in our protocol. Since the hybrid simulation is efficiently computable given the transcripts, it follows from the special zero-knowledge property of the underlying Fiat–Shamir protocol—and a standard hybrid argument reducing the indistinguishability of  $r$  executions to a single one—that our construction inherits the zero-knowledge property.

## 12 Zero-Knowledge Simulator: Precise Construction and Proof

Below we give (1) a clear, algorithmic description of the simulator  $Z = (Z_0, Z_1)$  (with English explanation of each step), (2) a hybrid-game argument that reduces the indistinguishability of the real interaction to the special zero-knowledge property of the underlying Fiat–Shamir (FS) protocol, and (3) an analysis of the two “bad events” — **InconsistentOracle** and **AmbiguousResponse** — with concrete bounds showing both are negligible. From this the full zero-knowledge claim follows.

Throughout:

- $k$  is the security parameter.
- $P$  is the honest prover for the compiled protocol;  $V$  the honest verifier.
- The FS subprotocol has: commitments **com** (length large, say  $\kappa$  bits), challenges of  $t$  bits, responses of  $b$  bits. The underlying FS protocol is assumed to be *special zero-knowledge* (there exists a simulator  $Z_{\text{FS}}$  that, on input  $(x, \text{ch})$ , outputs accepting transcripts  $(\text{com}, \text{ch}, \text{resp})$  indistinguishable from real ones) and to have the *unique-response* or special soundness property stated below.
- The compiled Fiat–Shamir protocol repeats the FS instance  $r = r(k)$  times (typically  $r = O(\log k)$ ), and the overall random oracles are modeled as an ideal random oracle  $H$  which the simulator controls.
- The adversary/distinguisher is  $\mathcal{D} = (\mathcal{D}_0, \mathcal{D}_1)$  (stage-wise), making at most  $Q(k)$  oracle queries.

### 12.1 The Simulator $Z = (Z_0, Z_1)$

#### Notation

- Oracle queries are of the form  $H(\cdot)$ . In the compiled protocol the hash is queried on tuples like  $(x, \text{com}, i, \text{ch}, \text{resp})$ .
- $\tau_i : \{0, 1\}^t \rightarrow \{0, 1\}^b$  denotes the simulator’s internal per-instance mapping.
- $Z_{\text{FS}}(x, \text{ch})$  denotes the special FS simulator producing  $(\text{com}, \text{ch}, \text{resp})$ .

#### 12.1.1 $Z_0(\cdot)$ — First Stage

#### Pseudocode

1. Initialize an empty table  $T_0$  representing the random oracle answers.
2. For every query  $q$  made by  $\mathcal{D}_0$ :
  - If  $q \in T_0$ , return  $T_0[q]$ .
  - Else sample  $y \leftarrow \{0, 1\}^b$ , set  $T_0[q] = y$ , return  $y$ .
3. Return to  $\mathcal{D}_0$  the simulated first-stage view (including the oracle implemented by  $T_0$ ).

**Explanation**  $Z_0$  simply implements a uniformly random oracle. No programming occurs yet.

### 12.1.2 $Z_1(x)$ — Second Stage

#### Pseudocode

1. For  $i = 1$  to  $r$ :
  - (a) Sample independently  $2^t$  uniform  $b$ -bit strings to form a temporary mapping  $\tau_i^{\text{temp}}$ .
  - (b) Let  $ch_i$  be the lexicographically minimal challenge achieving the minimal output of  $\tau_i^{\text{temp}}$ .
  - (c) Run  $Z_{\text{FS}}(x, ch_i)$  to obtain  $(\text{com}_i, ch_i, \text{resp}_i)$ .
  - (d) Set  $\tau_i(ch_i) := \tau_i^{\text{temp}}(ch_i)$ .
2. Copy  $T_0$  to  $T_1$ .
3. For each  $i$ :
  - For the tuple  $q_i = (x, \text{com}_i, i, ch_i, \text{resp}_i)$  satisfying the verification predicate, set  $T_1[q_i] := \tau_i(ch_i)$ .
4. When  $\mathcal{D}_1$  makes queries:
  - If  $q \in T_1$  return  $T_1[q]$ .
  - Else sample  $y \leftarrow \{0, 1\}^b$ , set  $T_1[q] = y$ , return  $y$ .
5. Output  $\pi = (\text{com}_i, ch_i, \text{resp}_i)_{i=1}^r$ .

**Explanation** Each repetition uses the special FS simulator with a chosen challenge  $ch_i$  and programs the oracle only on the tuple needed for consistency.

## 13 Hybrid Reduction

We compare the following experiments observed by  $\mathcal{D}$ .

- **Real:** Real execution with genuine FS transcripts and a true random oracle.
- **Hybrid<sub>j,j=0...r</sub>:** *The first  $j$  FS instances are simulated by  $Z_{\text{FS}}$ , the rest are real.*
- **Sim:** Output of  $Z$  (all FS instances simulated and oracle programmed).

### 13.1 Hybrid Argument

For each  $j$ , distinguishing Hybrid <sub>$j-1$</sub>  from Hybrid <sub>$j$</sub>  gives a distinguisher for the FS special zero-knowledge property. Thus:

$$|\Pr[\mathcal{D} \text{ outputs } 1 \mid \text{Hybrid}_j] - \Pr[\mathcal{D} \text{ outputs } 1 \mid \text{Hybrid}_{j-1}]| \leq \epsilon_{\text{FS}}(k).$$

Summing over  $j$  yields that replacing all  $r$  transcripts by simulated ones changes the distribution by at most  $r \cdot \epsilon_{\text{FS}}(k)$ , negligible.

Thus the only remaining difference from **Sim** is oracle programming, controlled by two bad events.

## 14 Bad Event 1: InconsistentOracle

### 14.1 Definition

Let

$$q_i := (x, \text{com}_i, i, \text{ch}_i, \text{resp}_i)$$

be the programmed tuple. **InconsistentOracle** occurs if  $\mathcal{D}_0$  queried  $q_i$  during stage 1 and received a value *different* from  $\tau_i(\text{ch}_i)$ , causing a conflict.

### 14.2 Bounding the Probability

- $\mathcal{D}_0$  makes at most  $Q(k)$  queries.
- For each  $i$ ,  $\text{com}_i$  has entropy  $\kappa(k)$  (exponentially large).
- Probability that any of stage-1 queries equals  $q_i$  is at most  $Q(k)/2^{\kappa(k)}$ .
- Union bound over  $r$  instances:

$$\Pr[\text{InconsistentOracle}] \leq \frac{r \cdot Q(k)}{2^{\kappa(k)}}.$$

Since  $\kappa(k)$  is large, this is negligible.

## 15 Bad Event 2: AmbiguousResponse

### 15.1 Definition

Given  $(\text{com}, ch)$ , **AmbiguousResponse** occurs if the adversary produces *two distinct* accepting responses  $\text{resp} \neq \text{resp}'$  for the same challenge, but both map to the same oracle value under the simulator's programming.

### 15.2 Bounding the Probability

- In the FS protocol (or underlying  $\Sigma$ -protocol), for fixed  $(\text{com}, ch)$  there is at most one accepting response.
- Two distinct accepting responses contradict special soundness and would allow extraction of the witness.
- Therefore producing two such responses has negligible probability.

Thus  $\Pr[\text{AmbiguousResponse}]$  is negligible.

## 16 Putting Everything Together

We have:

$$\text{Adv}_{\mathcal{D}}^{\text{ZK}} \leq r \cdot \epsilon_{\text{FS}}(k) + \Pr[\text{InconsistentOracle}] + \Pr[\text{AmbiguousResponse}],$$

each negligible, hence the total is negligible.

Therefore the compiled Fiat–Shamir protocol is zero-knowledge.

## 17 Additional Remarks

- Choosing the minimal challenge  $ch_i$  ensures a canonical rule independent of the adversary.
- High-entropy commitments guarantee unpredictability of  $q_i$  during stage 1.
- The simulator achieves standard negligible bounds.

## 18 Summary

- Constructed an explicit simulator  $Z = (Z_0, Z_1)$ .
- Showed indistinguishability using a standard hybrid argument.
- Analyzed and bounded the two bad events.
- Concluded that the simulator's output is computationally indistinguishable from the real prover's output.

### 18.1 Online Extraction

In this section we present the core feature of our construction: an *online* knowledge extractor that is able to extract a witness from any malicious prover *without rewinding*, unlike classical Fiat–Shamir (FS) proofs. Recall that when the prover is honest the extractor is not required; extraction is defined only for a malicious prover who convinces the verifier of a false statement. The extractor must guarantee that whenever such a prover convinces the verifier with non-negligible probability, it is possible to extract a witness. In other words: *without a valid witness, the verifier's acceptance probability must be negligible*.

The advantage of our construction over traditional rewinding-based FS proofs is that it enables us to obtain, *online*, two accepting transcripts of the form

$$(x, \text{com}, i, \text{ch}_i, \text{resp}_i) \quad \text{and} \quad (x, \text{com}, i, \text{ch}'_i, \text{resp}'_i),$$

for some repetition index  $i$  and  $\text{ch}_i \neq \text{ch}'_i$ , both of which satisfy the FS verification predicate. Once such a pair of accepting transcripts is obtained, the standard knowledge extractor for the underlying FS protocol (denoted  $K_{\text{FS}}$ ) can be invoked to extract the witness.

#### Setup

Let  $A$  be a malicious prover interacting with a random oracle  $H$ . Its execution proceeds as:

- $H$  is a random oracle.
- $(x, \pi) \leftarrow A^H(k)$ , where  $x$  is the statement and  $\pi$  is the proof output by  $A$ .
- $Q_H(A)$  denotes the sequence of queries made by  $A$  to  $H$  during the computation of  $\pi$ .
- The online extractor  $K$  receives  $(x, \pi, Q_H(A))$  and must output a witness  $w$  with overwhelming probability.

The extractor  $K$  succeeds whenever it finds two accepting oracle queries sharing the same prefix  $(x, \text{com}, i)$  but with distinct challenges:

$$(x, \text{com}, i, \text{ch}_i, \text{resp}_i), \quad (x, \text{com}, i, \text{ch}'_i, \text{resp}'_i), \quad \text{ch}_i \neq \text{ch}'_i.$$

Thus it suffices to bound the probability that *no such pair exists* for any repetition index  $i$  while the verifier still accepts the proof  $\pi$ .

### Analysis for a Fixed Prefix

Fix a particular tuple  $(x, \widetilde{\text{com}})$  and a repetition index  $i$ . By assumption, adversary  $A$  *never* makes two oracle queries of the form

$$(x, \widetilde{\text{com}}, i, \text{ch}_i, \text{resp}_i) \quad \text{and} \quad (x, \widetilde{\text{com}}, i, \text{ch}'_i, \text{resp}'_i)$$

with  $\text{ch}_i \neq \text{ch}'_i$ , both accepted by  $V_{\text{FS}}$ . If this occurred, the online extractor would immediately obtain the witness.

Similarly,  $A$  cannot query two distinct accepting responses for the same challenge:

$$(x, \widetilde{\text{com}}, i, \text{ch}_i, \text{resp}_i), \quad (x, \widetilde{\text{com}}, i, \text{ch}_i, \text{resp}'_i), \quad \text{resp}_i \neq \text{resp}'_i,$$

since this would violate the *unique-response* property of the FS protocol. This event can occur only with negligible probability.

Thus, for each repetition  $i$  and each prefix  $(x, \widetilde{\text{com}})$ , we can associate a single value

$$s_i := H(x, \widetilde{\text{com}}, i, \text{ch}_i, \text{resp}_i)$$

if  $A$  queries such a tuple; otherwise,  $s_i$  is a fresh uniform  $b$ -bit value. Therefore, the values  $s_1, \dots, s_r$  are mutually independent and uniformly random.

### Bounding the Probability of Passing Verification Without Extraction

Let  $S$  be the acceptance threshold of the verifier. We now compute the probability that the sum

$$T := s_1 + s_2 + \dots + s_r$$

falls below  $S$ , even though no repetition index  $i$  produced two distinct valid queries (i.e., no pair revealing the witness).

The number of ways to write a sum  $T \leq S$  as

$$T = s_1 + \dots + s_r, \quad s_i \geq 0,$$

is at most

$$\sum_{T=0}^S \binom{T+r-1}{r-1} \leq (S+1) \cdot \binom{S+r}{r-1} \leq (S+1) \left( \frac{e(S+r)}{r-1} \right)^{r-1}.$$

Since  $S = O(r)$ , there exists a constant  $c$  such that  $S + r \leq c(r - 1)$ , and

$$(S + 1) \left( \frac{e(S + r)}{r - 1} \right)^{r-1} \leq (S + 1) \cdot 2^{(\log(ec))r},$$

which is polynomial in  $k$ .

On the other hand, the random variables  $s_1, \dots, s_r$  are independently uniform in  $\{0, 1\}^b$ , so the total number of possible tuples is  $2^{br}$ . Therefore,

$$\Pr[T \leq S] \leq \frac{(S + 1) \cdot 2^{(\log(ec))r}}{2^{br}},$$

which is negligible for the chosen parameters.

### Extending to All Possible Prefixes

There are at most  $Q + 1$  possible prefixes  $(x, \widetilde{\text{com}})$  arising from the  $Q$  hash queries made by  $A$ . Applying a union bound, the probability that  $A$  produces an accepting proof *and* that the extractor fails for all such prefixes is at most

$$(Q + 1)(S + 1) \cdot 2^{(\log(ec) - b)r},$$

which remains negligible.

□

## 19 Simulation soundness

**Theorem 3** (Multiple zero-knowledge and simulation soundness). *Let  $(P_{\text{FS}}, V_{\text{FS}})$  be an interactive Fiat–Shamir proof of knowledge for relation  $W$ , and let  $(P, V)$  be the non-interactive protocol derived from Construction 1. Then the following hold:*

1. (Multiple zero-knowledge) *There exists a PPT simulator  $Z$  such that for every PPT distinguisher  $D$  the two experiments  $\text{Real}^D$  and  $\text{Sim}^D$ , where  $D$  interacts with  $n$  (possibly adaptively chosen) theorems and proofs, are computationally indistinguishable.*
2. (Simulation soundness) *There exists a PPT online extractor  $K$  such that for every PPT adversary  $B$  that obtains simulated proofs for adaptively chosen theorems, the probability that  $B$  outputs  $(x, \pi)$  with  $V^{H_n}(x, \pi) = 1$ , yet  $K$  on input  $(x, \pi, Q_H^B)$  fails to output a witness  $w$  with  $(x, w) \in W_k$ , is negligible in the security parameter.*

*Proof.* We only give the proof structure; the detailed arguments must be filled in from the original source.

## Multiple zero-knowledge.

**Experiments.** Define the “real” experiment in which  $D$  interacts with a true random oracle  $H$  and honest prover  $P^H$ , answering  $n$  (adaptive) theorem queries  $x_i$  with real proofs  $\pi_i$ . Define the “simulated” experiment in which  $D$  interacts with the multi-theorem simulator  $Z = (Z_0, Z_1, \dots, Z_n)$ , which maintains oracles  $H_i$  and simulated proofs  $\pi_i$ .

**Simulator construction.** •  $Z_0$  samples an initial oracle  $H_0$  and state  $\tau_0$ .  
• For each query  $i = 1, \dots, n$ , given  $(x_i, \tau_{i-1})$ , the simulator  $Z_i$  outputs  $(H_i, \pi_i, \tau_i)$ , where  $H_i$  extends  $H_{i-1}$  consistently and  $\pi_i$  is a proof for  $x_i$  (or a simulated failure symbol if  $x_i \notin L$ ).

**Hybrid sequence.** Introduce hybrids that:

1. Replace the  $i$ -th real proof by a single-theorem simulation using the (special) zero-knowledge simulator for the underlying Fiat–Shamir protocol.
2. Argue indistinguishability between consecutive hybrids via the single-theorem ZK property and a standard hybrid argument over  $i = 1, \dots, n$ .

**Oracle consistency and uniqueness of commitments.** Use the super-logarithmic min-entropy of commitments and the “unique responses” property to bound:

- The probability that  $D$  has previously queried  $H$  on a commitment that will later appear in a simulated proof (event **InconsistentOracle**).
- The probability that two distinct accepting transcripts for the same  $(x, \text{com}, i, \text{ch})$  exist (event **AmbiguousResponse**).

Both probabilities are negligible; condition on their complement.

## Simulation soundness.

**Goal.** Show that the *same* online extractor  $K$  as in the single-theorem case suffices even when the adversary  $B$  has seen arbitrarily many simulated proofs  $(x_i, \pi_i)$  from  $Z$ .

**Assume towards contradiction.** Suppose there exists a PPT  $B$  such that, in the multi-theorem simulation experiment,  $B$  outputs a valid  $(x, \pi)$  with  $V^{H_n}(x, \pi) = 1$ , but  $K(x, \pi, Q_H^B)$  fails to output a valid witness with non-negligible probability.

**Reduction to basic online extraction.** Construct an adversary  $A$  for the *basic* (no prior proofs) online extraction experiment for  $(P, V)$ :

- $A$  internally runs  $B$  and uses the real prover  $P^H$  to answer all of  $B$ ’s proof queries  $(x_i, \pi_i)$ , relaying hash queries between  $B$  and the random oracle.
- When  $B$  outputs its final  $(x, \pi)$ ,  $A$  outputs the same pair in the basic extraction experiment.

Argue that the distribution of  $(x, \pi, Q_H^B)$  in the basic experiment (with  $A$ ) is indistinguishable from that in the simulated one (with  $B$  and  $Z$ ), by the multiple ZK property.

**Extractor's view and irrelevant queries.** Observe that  $K$  only needs hash queries with prefix  $(x, \text{com})$  for the final theorem  $x$ ; hash queries related to other theorems  $(x_i, \text{com}_i)$  are irrelevant for extraction and can be dropped without decreasing the success probability.

**Contradiction.** Since the extraction error in the basic setting is negligible, the extraction error in the simulated multi-theorem setting must also be negligible (else distinguishers could separate the two views, breaking multiple zero-knowledge). This contradicts the assumed non-negligible failure probability of  $K$  against  $B$ .

□